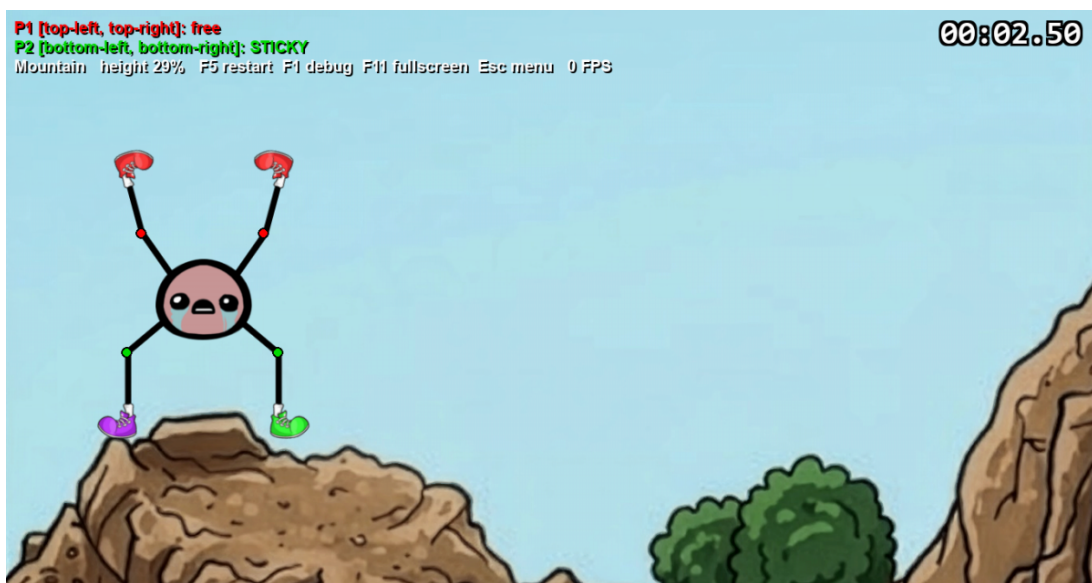


All For One

Code Guide

How every part of the game works - structure, data flow, physics, vision, and every file explained block by block with its full source



From The Group MeEeEe - ME461

03 October 2026

Contents

Part I - The big picture	4
1. What the game is	4
2. Running it	4
3. The files	5
4. How the parts fit together	5
5. One frame, start to finish	6
6. Coordinates, units and conventions	7
Part II - The core systems	9
7. Terrain and collision: the signed distance field	9
8. Physics of the character	9
9. From the camera to the legs	11
10. Drawing	13
11. The screens	14
Part III - Every file, block by block	16
main.py (44 lines)	16
app.py (51 lines)	17
config.py (155 lines)	18
display.py (120 lines)	21
menu.py (144 lines)	23
ui.py (77 lines)	26
game.py (207 lines)	28
test_scene.py (77 lines)	32
maps.py (59 lines)	34
terrain.py (96 lines)	36
flag.py (47 lines)	38
viewport.py (44 lines)	39
character.py (276 lines)	40
assets.py (110 lines)	46
inputs.py (67 lines)	48
vision.py (381 lines)	50
player_state.py (108 lines)	56
preview.py (85 lines)	58
export_engine.py (17 lines)	60
Part IV - Reference	61
12. Every setting in config.py	61
13. How do I...?	63

14. Glossary	64
Part V - How the AI models work	65
15. Pose detection: YOLO	65
16. Face landmarks: MediaPipe Face Landmarker	65
17. Limits to keep in mind	66

Part I - The big picture

1. What the game is

All For One is a co-op climbing game. Up to four people stand in front of one webcam and together control one cartoon character with four legs. Every player's arm (shoulder and elbow) drives one of the character's legs (hip and knee). There are no buttons: you push the character up the mountain by moving your arms, and you make its shoes sticky by opening your mouth. The goal is to touch the flag with the character's head as fast as possible.



The game window: the game view on the left, the camera picture on the right. P2's mouth is open, so the bottom shoes are sticky - purple = glued to the rock, green = sticky but still in the air.

Behind that simple idea there are four separate problems, and the code is organised around them:

- **Seeing the players** - find every person's body joints and mouth in the webcam picture (`vision.py`, using two AI models).
- **Turning bodies into commands** - convert arm positions into leg angles and mouth openings into sticky feet (`player_state.py`, `game.py`).
- **Physics** - make a character with rigid legs fall, stand, walk, jump and climb on an arbitrary drawn landscape (`terrain.py`, `character.py`).
- **Showing it all** - menus, the game view that follows the character, the camera preview, fullscreen (`display.py`, `menu.py`, `game.py`, `preview.py`).

2. Running it

Command	What it does
<code>python main.py</code>	Normal game with webcam 0.
<code>python main.py --source 1</code>	Use another webcam (or a video file path).
<code>python main.py --keyboard</code>	No camera: legs move with Q/A W/S E/D R/F U/J I/K O/L P/; keys, hold 1-4 to open a player's mouth. Good for testing physics.
<code>python main.py --debug</code>	Start with collision circles shown (same as pressing F1).
<code>python export_engine.py</code>	Build the TensorRT engine once per computer (NVIDIA GPU only) for about 2x faster pose detection.

In the game: **F11** fullscreen, **F5** restart, **F1** collision circles, **Esc** back to the menu.

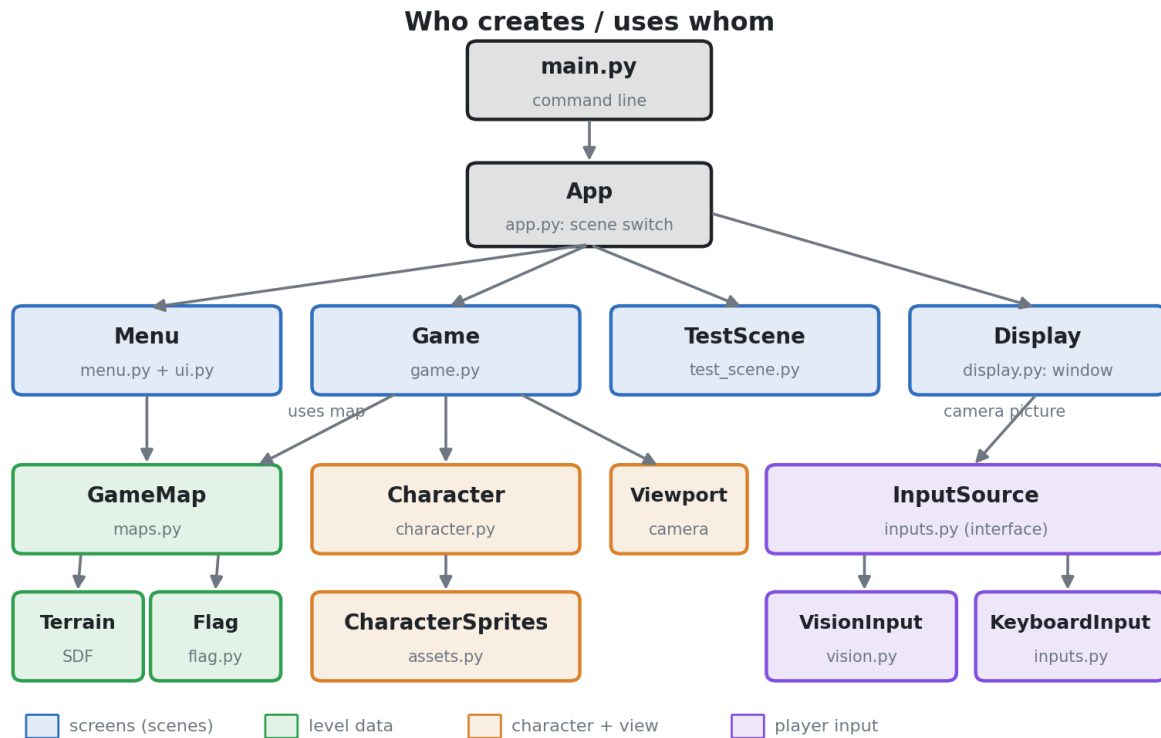
3. The files

Every file has one job (the assignment asked for separate files per task and object-oriented code). The right column says which class or function to look for.

File	Job	Main contents
main.py	Entry point: read the command line, choose camera or keyboard	<code>main()</code>
app.py	Create the window and switch between the screens	<code>App</code>
config.py	Every tunable number in one place	constants
display.py	The one window: game panel + camera panel, scaling, fullscreen	<code>Display</code>
menu.py	Title page, player-count page, map page	<code>Menu</code>
ui.py	Buttons and outlined text	<code>Button</code> , <code>MapButton</code> , <code>draw_text</code>
game.py	The level: loop, physics steps, HUD, timer, finish	<code>Game</code> , <code>apply_players</code>
test_scene.py	1-player test screen (no physics)	<code>TestScene</code>
maps.py	Load a map: scale it, find start point and flag	<code>GameMap</code> , <code>load_maps</code>
terrain.py	Map picture + collision (signed distance field)	<code>Terrain</code>
flag.py	The goal flag: placement, touch test, drawing	<code>Flag</code>
viewport.py	The game camera that follows the character	<code>Viewport</code> , <code>view_size</code>
character.py	Legs, kinematics, physics, drawing of the character	<code>Leg</code> , <code>Character</code>
assets.py	Character pictures: crop, scale, stretch, recolour, rotate	<code>PivotSprite</code> , <code>CharacterSprites</code>
inputs.py	Input interface + keyboard input	<code>InputSource</code> , <code>KeyboardInput</code>
vision.py	Webcam thread: pose, faces, players, mouth, head tilt	<code>VisionInput</code> and helpers
player_state.py	Player data + arm-angle maths	<code>PlayerState</code> , <code>limb_poses</code>
preview.py	Draws skeletons and lips on the camera picture	<code>PreviewRenderer</code>
export_engine.py	Builds the TensorRT engine	script
pose_and_face_test.py	Your original pose + face experiment (not used by the game)	script

The `assets/` folder holds the pictures: `background*.png` (sky and far scenery), `foreground*.png` (the rock you collide with - its transparent pixels are air), the character parts `head.png`, `leg-part.png`, `leg-part-with-shoe.png`, `character-demo.png` (menu picture) and `flag.png`. The two AI model files `yolo26n-pose.pt` and `face_landmarker.task` download themselves on first run.

4. How the parts fit together



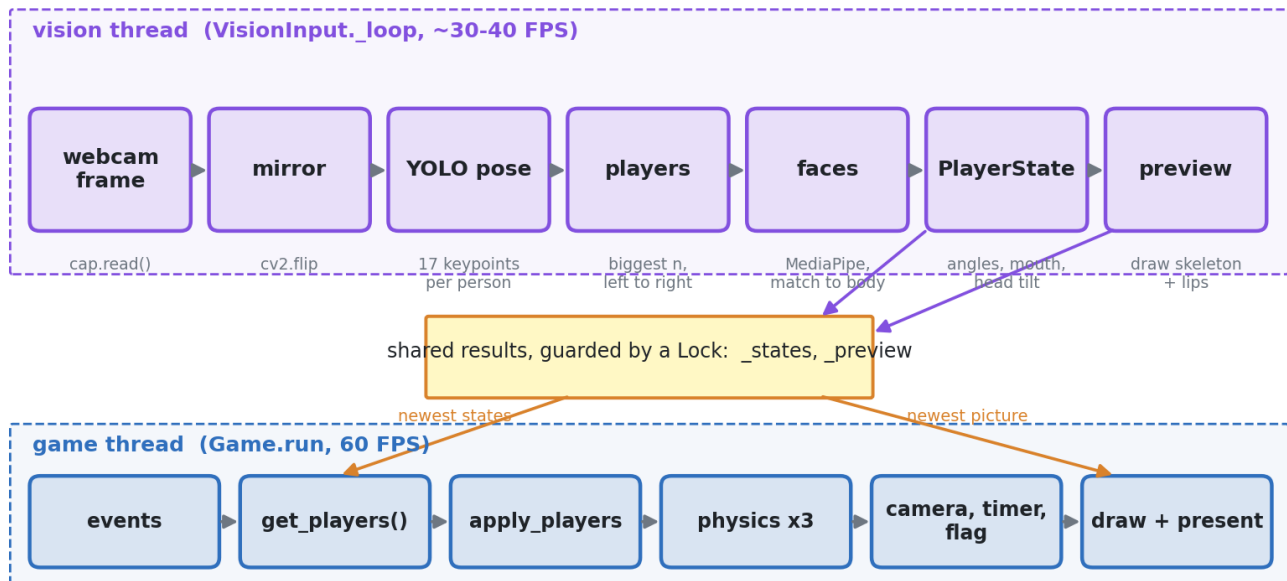
Who creates and uses whom. Arrows point from the user to the thing it uses.

Reading the diagram top-down: `main.py` creates an **input source** (camera or keyboard) and gives it to `App`. `App` creates the window (`Display`), loads the maps and creates the three **scenes** (`Menu`, `Game`, `TestScene`). A scene is a screen with its own loop. `App.run()` asks the menu what to do, runs that scene until it returns, and goes back to the menu.

The game scene uses a `GameMap` (which owns a `Terrain` for collisions and a `Flag`), builds a `Character` (which uses `CharacterSprites` for its pictures) and a `Viewport` (the game camera). It never talks to the webcam directly - it only calls `input.get_players()`. That is the key design idea: **the game does not know whether a camera or a keyboard is behind the input**. Both classes inherit from `InputSource` and offer the same methods (`get_players`, `get_preview`, `set_num_players`, `close`). This is called an *interface*, and it is why keyboard mode was trivial to add.

5. One frame, start to finish

Two things run at the same time (two *threads*). The **vision thread** reads the webcam and runs the AI models as fast as it can (about 30-40 times per second). The **game thread** runs the game at a steady 60 frames per second. They meet in one place: a small shared box with the newest results, protected by a *lock* so the game never reads a result while the vision thread is half-way through writing it.



The two loops and the shared results between them.

Why two threads? Running the AI models takes 20-110 ms per picture. If the game waited for them every frame, it would stutter at 9-30 FPS. With a separate thread the game always runs smoothly and simply uses the newest available result.

Start-up, in order

1. `main()` parses the command line and creates `VisionInput` - this loads the pose model (picking TensorRT, GPU or CPU), the face model, opens the webcam and starts the vision thread.
2. `App.__init__` starts pygame and creates the `Display` (the window).
3. `load_maps()` loads every map in `config.MAPS`: scales the pictures, computes the collision field, places the flag and finds the start point (about 2 seconds in total).
4. The three scenes are created. `App.run()` shows the menu.
5. The player picks PLAY, a player count and a map; `Game.run(players, map)` starts the level.

Every game frame (Game.run)

1. `clock.tick(60)` waits so frames are 1/60 s apart and returns the real time since the last frame (`dt`).
2. Keyboard/window events: Esc, F5, F1, F11, closing the window.
3. `input.get_players()` returns one `PlayerState` per player (limb angles, mouth, head tilt).
4. `apply_players` copies each controlling limb's angles into its leg as a *target*, and sets each leg's *sticky* flag from the mouth.
5. Physics: `character.update()` three times with `dt/3` each (substeps).
6. The viewport glides towards the character; the timer runs; the flag touch is checked.
7. Drawing: map part, flag, character, knee dots, texts, timer, and `display.present()` shows it.

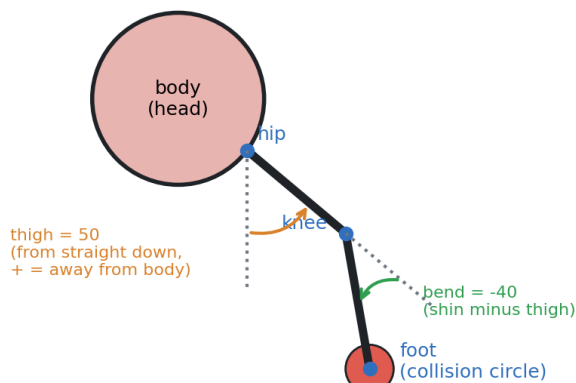
6. Coordinates, units and conventions

Most confusion in game code comes from coordinates. These are the conventions used everywhere here:

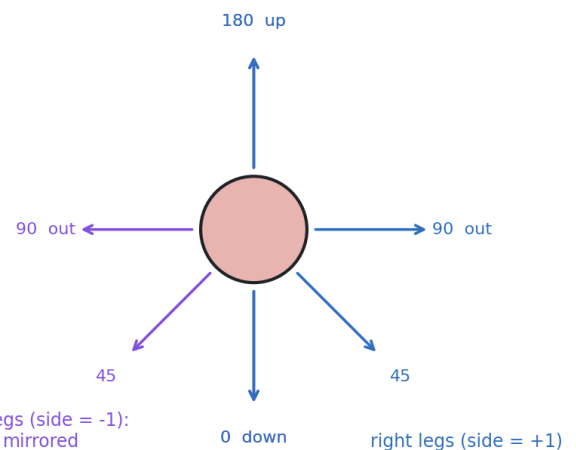
- **Map pixels** are the unit of the world. All maps are scaled so the mountain map is 2164 px tall (`MAP_HEIGHT`); positions, speeds (px/s) and gravity (px/s²) are in map pixels.

- **y points down** (screen convention). Gravity is **+y**; "up" means a smaller y. The HUD height % is computed as `map height - y`.
- **Three sizes of picture**: the *map* (e.g. 4096 x 2164), the *canvas* (1365 x 721, the size the scenes draw at = what the viewport shows) and the *window* (whatever fits your screen). `display.present()` scales canvas to window; `display.to_canvas()` converts mouse clicks back.
- **Screen position = map position - viewport position**. That is why every `draw()` gets an `offset`.
- **Angles are degrees**. Leg and arm angles use the *outward* convention below.
- **Colours**: pygame uses (R, G, B), OpenCV uses (B, G, R). `config.PLAYER_COLORS_BGR` is the master list and the RGB list is made by reversing each tuple.

right leg: angles in the 'outward' convention



the same thigh angle points outward on both sides



Left: a leg is two angles. Right: left legs are mirrored so 'away from the body' is positive on both sides.

A leg is fully described by two numbers: **thigh** = the hip angle measured from straight down (0 = hanging, 90 = sideways, 180 = up) and **bend** = how much the shin turns relative to the thigh. The *same* two numbers are measured on the player's arm (shoulder angle and elbow angle), which is why an arm can drive a leg directly: arm hanging down -> leg hanging down, arm raised -> leg raised.

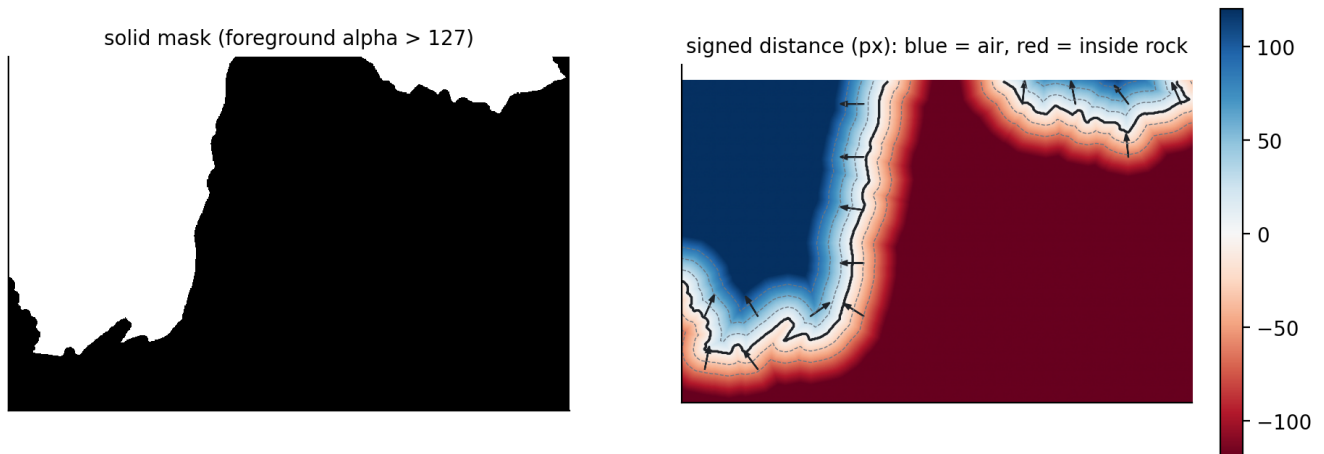
To draw or compute a leg on screen, the outward angle is multiplied by the leg's **side** (-1 left, +1 right) to get a normal screen rotation (`Leg.screen_angles`).

Part II - The core systems

7. Terrain and collision: the signed distance field

The landscape is a drawing, not a set of boxes or polygons, so collision must work on pixels. The foreground picture's transparency decides what is solid: a pixel more than half opaque is rock.

Terrain turns that yes/no mask into a **signed distance field (SDF)**: a table with one number per pixel telling how far that pixel is from the nearest rock surface - positive in the air, negative inside rock, zero on the surface.



A part of the mountain. Right: the distance field; the black line is distance 0 (the surface), dashed lines are every 30 px, arrows are the normals (the way out).

OpenCV computes this table once per map with `cv2.distanceTransform` (run on the air and on the rock, then subtracted). After that, three questions are cheap lookups:

- **Is this point inside rock, and how deep?** `terrain.distance(x, y)`. A foot circle of radius r is in the ground when `distance < r`, and it is $r - \text{distance}$ deep.
- **Which way is out?** `terrain.normal(x, y)` - the direction in which the distance grows fastest (its gradient), found by sampling the distance 2 px left/right and up/down.
- **Where is the ground under the sky in this column?** `terrain.ground_y(x)`, used to place the flag.

```
distance(x, y) = bilinear blend of the 4 table cells around (x, y)
normal = normalize( d(x+2,y) - d(x-2,y) , d(x,y+2) - d(x,y-2) )
```

Bilinear blending means the distance changes smoothly between pixels instead of in 1 px steps, which keeps the physics from jittering. Points outside the map are handled specially: beyond the left and right edges counts as ever-deeper rock (invisible walls), and above the top edge the top row continues upward - sky stays sky, but a solid top border (like the cave's) becomes rock that gets deeper going up, so nothing can escape through it.

8. Physics of the character

The character is deliberately simple: **only the body (the head) has a position and a velocity**. It cannot rotate. The legs have no mass - each leg is just two angles that turn towards the angles the player asks for, at most 420 degrees per second. Where a foot is follows from the body position plus the two angles (*kinematics*, `Leg.joints`):

```

hip = body + hip_offset
knee = hip + rotate((0, thigh_length), thigh angle)
foot = knee + rotate(foot_offset, shin angle) # shin angle = thigh + bend

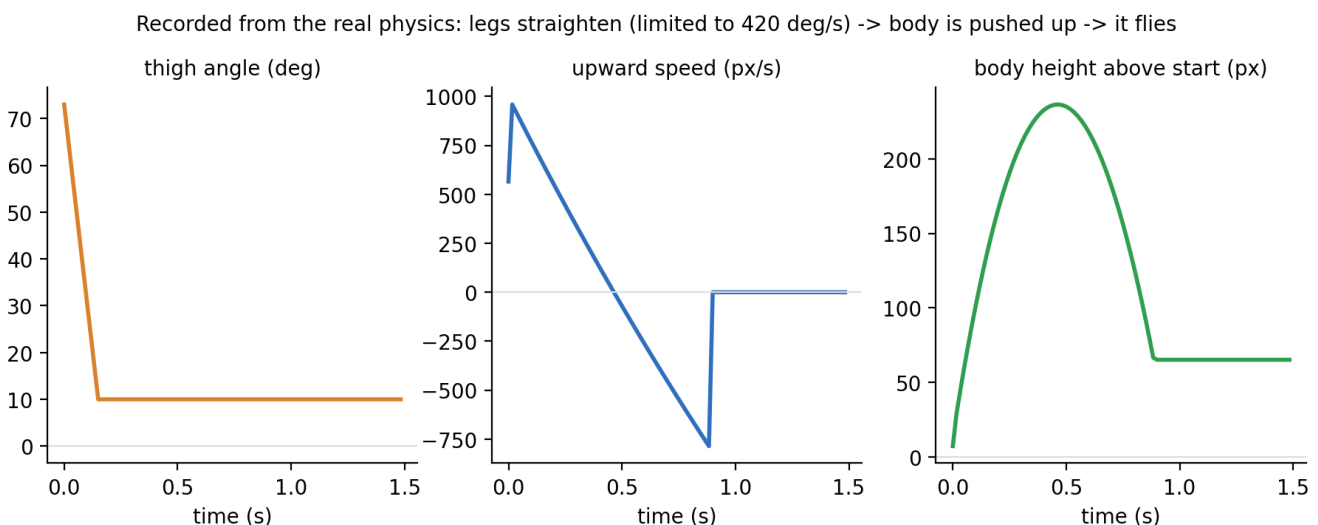
```

The method is **position-based dynamics** (PBD): instead of computing forces, each step first moves the body freely, then *corrects its position* until all contacts are satisfied, and finally derives the velocity from how far the body actually moved. One `_step` does:

1. **Sticky feet** - a sticky foot that is within 8 px of the rock gets *glued*: its position is remembered as an anchor. A foot that stops being sticky lets go.
2. **Predict** - add gravity to the velocity, apply a little air drag, move the body by velocity x dt.
3. **Solve** (4 passes) - for every foot: if glued, move the body so the foot goes back towards its anchor; otherwise, if the foot is inside rock, push the body out along the normal and apply friction. Then push the head out of the rock. Each correction can upset another, so the passes repeat (the Gauss-Seidel method).
4. **Rock has the last word** - two extra passes make sure no glued foot is more than 6 px deep.
5. **Velocity** - `velocity = (new position - old position) / dt`, then limits (max speed, head friction, stay inside the map).

Why this gives jumping and walking for free

Jumping: if a planted leg straightens quickly, its foot would go into the ground. The solver pushes the body up instead, by exactly the amount the foot would have sunk. Because velocity is *measured* from the movement, the body now has upward speed and keeps flying after the foot leaves the ground. The recording below comes from the real physics:



A recorded jump on the mountain: the thigh angle drops (leg straightens), the body gets about 950 px/s upward, rises ~240 px and lands.

Walking: when a free foot first touches walkable ground (flatter than 55 degrees) it remembers that point (*grip*). While it stays down, the solver does not let the foot slide along the surface (*static friction*). So if the player sweeps the leg backwards, the foot cannot move - the body moves forward instead. On steeper slopes there is no grip and feet slide, which is what makes walls need sticky feet.

Climbing: a glued foot works in both directions - it can push *and pull* the body. Hanging from a ledge, a player can bend the glued leg to pull the body up. If the glue is stretched more than 40 px it tears off.

Keeping feet out of the rock while glued

With a foot glued, the body is pinned and cannot always move out of the way, so a leg could be forced into the rock. `Character.update` therefore checks every step: if a moving leg pushed its own foot deeper than allowed (3 px for free feet, 6 px for sticky feet), the step is undone and redone with that leg held still - the leg stops at the rock. A glued foot pushed in by *another* leg is held at 6 px by the extra solver passes.

Substeps

Physics runs three times per frame with a third of the time each. Smaller steps mean feet move less per step, so they cannot jump through thin rock, and the solver stays stable.

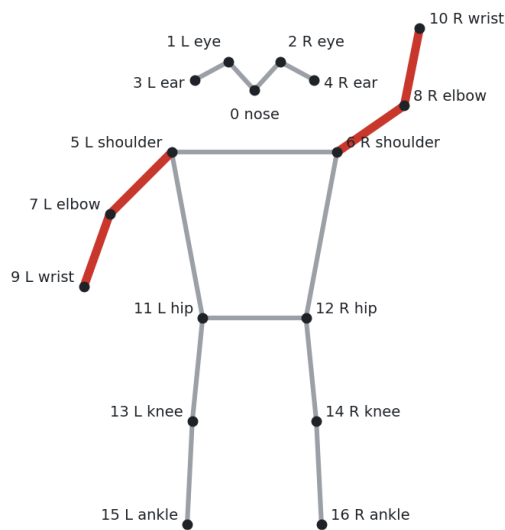
Setting	Value	Effect when you raise it
GRAVITY	2000 px/s ²	Heavier, falls faster, lower jumps
LEG_MAX_SPEED	420 deg/s	Legs react faster, kicks and jumps get stronger
MAX_SPEED	1500 px/s	Allows faster flights
WALKABLE_SLOPE	55 deg	Feet grip on steeper slopes without sticking
SOLVER_ITERATIONS	4	Stiffer, more exact contacts (slower)
STICKY_BREAK_DIST	40 px	Glue holds longer before tearing off
PHYSICS_SUBSTEPS	3	More stable, slower

9. From the camera to the legs

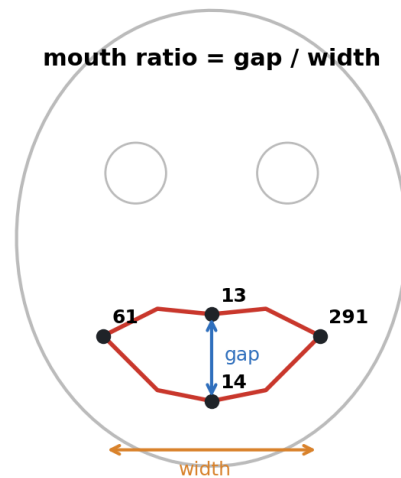
`VisionInput._loop` runs in the background thread. For each webcam frame it does:

1. **Mirror** the picture so it behaves like a mirror (your right hand appears on the right).
2. **Pose**: YOLO finds every person and 17 body keypoints each, with a confidence per keypoint.
3. **Number the players** (`assign_players`): take the n biggest people (closest to the camera) and number them left to right. With fewer people than players, the picture is split into n vertical strips and you get the number of the strip you stand in.
4. **Faces**: MediaPipe finds face landmarks on the whole frame. `match_faces` gives each face to the person whose head keypoints are closest in pixels (closest pairs first). A player without a face gets a second try on a crop around their head.
5. **Per player**: limb angles (`limb_poses`) smoothed by `AngleSmoother`, the mouth state (`MouthTracker`) and, with one player, the head tilt (`HeadTilt`).
6. **Preview**: draw the skeletons and lips on a copy of the frame (`PreviewRenderer`).
7. Store the results under the lock.

COCO 17 keypoints (YOLO pose output);
controlling arms coloured, rest grey



4 of the 478 face-mesh points used for the mouth



Left: the 17 keypoints of one body. Right: the four face points used for the mouth.

Arm angles

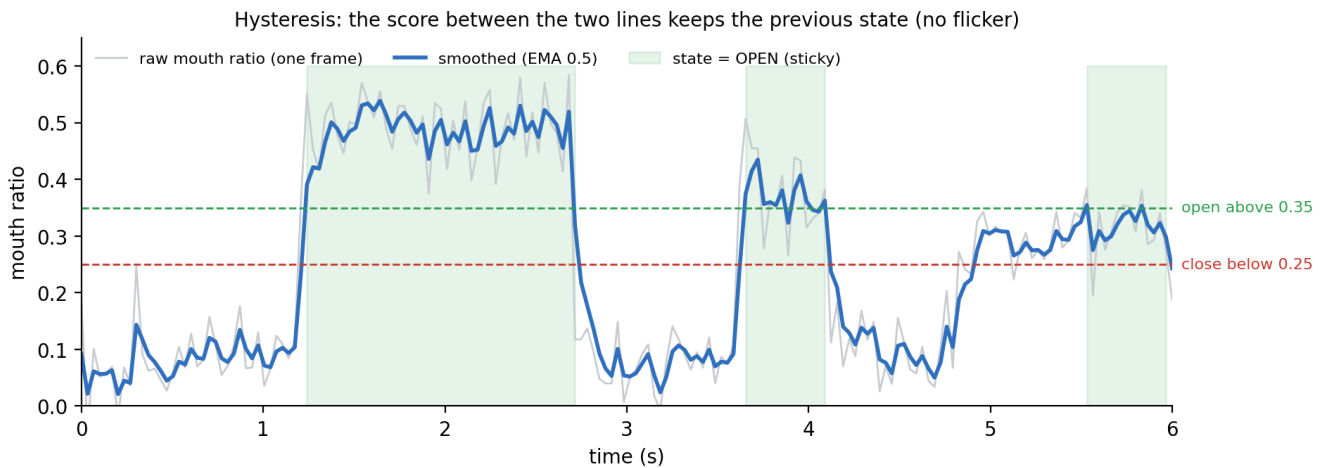
For an arm, the shoulder angle is the direction of shoulder -> elbow and the elbow angle is the direction of elbow -> wrist, both measured from 'down' and corrected by the torso direction (shoulders -> hips) so leaning does not swing the legs. Left limbs are mirrored (multiplied by -1) to get the outward convention. Sides come from where the joints are on screen, not from the model's left/right labels, because on a mirrored picture those can be swapped.

```
heading(v) = atan2(v.x, v.y) # angle from straight down
thigh = side * wrap(heading(elbow - shoulder) - torso)
bend = wrap(side * wrap(heading(wrist - elbow) - torso) - thigh)
```

Keypoints jitter by a few pixels every frame. **AngleSmoother** applies an *exponential moving average* (EMA): each new value moves the smoothed value 60% of the way. It is *wrap-safe*: going from 179 to -179 degrees is treated as a 2 degree step, not a 358 degree swing.

Mouth: open or closed

The mouth ratio is the gap between the lips divided by the mouth width. Dividing by the width makes it the same whether the face is near (big) or far (small). The ratio is smoothed, then a **hysteresis** decides the state: it opens only above 0.35 and closes only below 0.25. Between the two lines the state stays as it was, so a ratio hovering around one value cannot make the feet flicker.



Hysteresis on a made-up recording: the 0.30 bump at $t = 3.8$ s is not enough to open; noise around the thresholds does not toggle the state.

If the face is lost for a moment, the last state is kept for 0.5 s before it falls back to closed.

Head tilt (1 player)

With one player, both legs belong to the same mouth, so the sideways tilt of the head picks which foot the open mouth makes sticky. The roll angle is the slope of the line between the ears (or the eyes if an ear is hidden). Tilting right (right ear lower) beyond 10 degrees selects the right foot, tilting left the left foot; in between, the last choice is kept.

10. Drawing

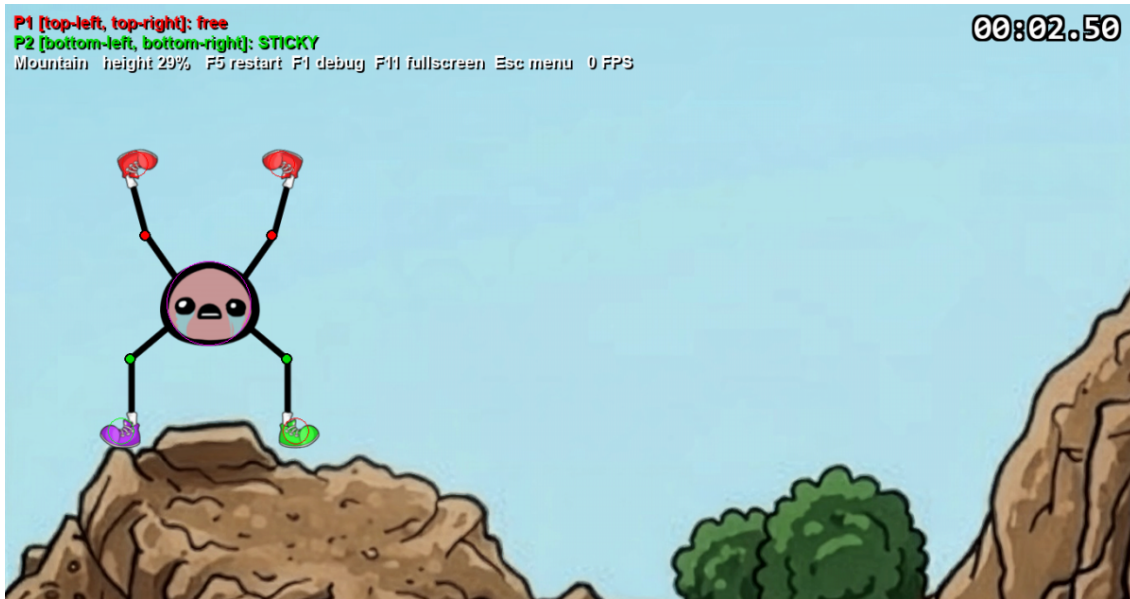
Sprites that rotate around a joint

All character pictures share one 500 x 500 canvas, and the joint positions were measured on it (`config.LEG_PIVOT` etc.). `PivotSprite.draw` rotates an image around its joint: pygame rotates around the image centre, so the code rotates the vector from the joint to the centre by the same angle and places the rotated image's centre there - the joint ends up exactly where it should.

Pictures are prepared once: cropped to their content (so rotating is cheap), scaled, mirrored for left legs (toes point outward) and recoloured. The recolour filter takes clearly red pixels and moves their 'redness' into other channels, which keeps the shading: green = sticky, purple = glued. For the 1-player long legs, `stretch_leg` stretches only the black stick part and moves the shoe down unchanged.

One window, two panels

Every scene draws onto the same canvas (1365 x 721). `Display.present()` fills the window, scales the canvas into the left panel, draws the newest camera picture into the right panel and calls `pygame.display.flip()`, which shows the finished frame all at once (no flicker). F11 recreates the window in fullscreen and recomputes the layout.



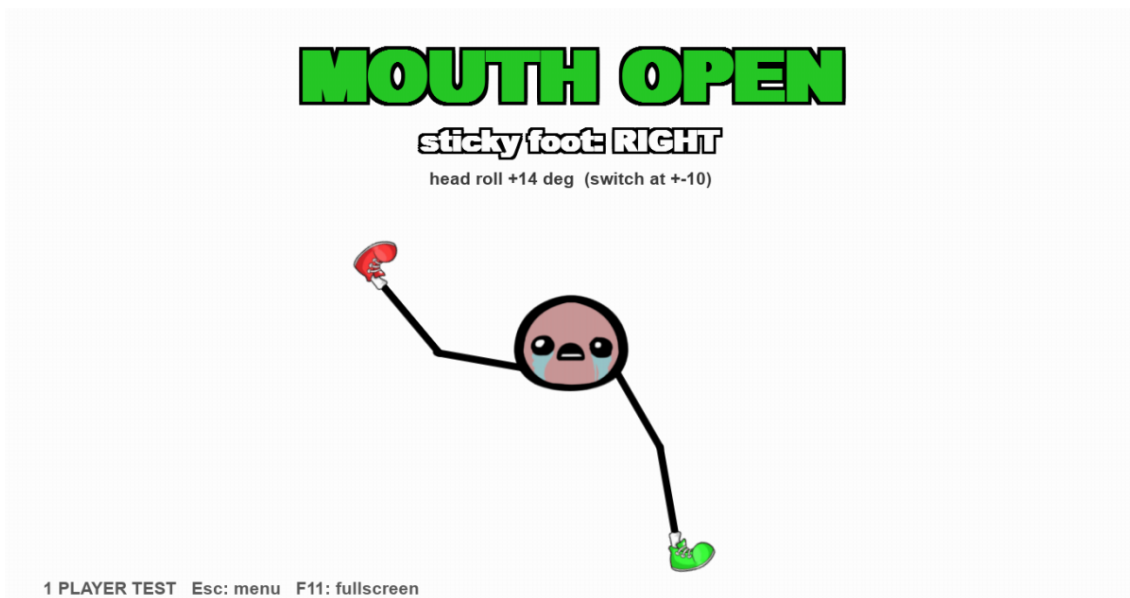
F1 debug view: magenta = head circle, feet circles coloured green (glued), yellow (touching) or red (in the air).

11. The screens

The menu has three pages in one loop (page = title, players, maps). PLAY leads to the player count, then to the map. 1P TEST opens the test screen. Every button position is a fraction of the canvas size.



The map page. Each picture is the whole map shrunk, with its flag.



The 1-player test screen: no physics, the body stays still and the legs follow the arms; mouth state in big letters.



Touching the flag with the head stops the timer.

Part III - Every file, block by block

Each file below starts with what it is for, then a table that walks through the file from top to bottom (the line numbers match the real file), then the complete source with its comments. Read the table next to the listing: the table says *why*, the comments in the code say *what*.

main.py (44 lines)

The entry point. It only reads the command line, creates the right input source and starts the app.

Lines	What happens there
1-12	Module docstring: how to start the program and what to install.
13-17	Imports. <code>KeyboardInput</code> is cheap to import; the camera input is imported later.
20-27	Declare the four command-line options; <code>parse_args</code> reads what the user typed.
29-37	Pick the input source. The vision module is imported only here because loading torch/ultralytics/mediapipe takes seconds.
38	Create the app with that source and run it until the player quits.
41-44	Standard Python idiom: run <code>main()</code> only when the file is started directly.

Full source

```

1  """All For One - entry point.
2
3      python main.py                # webcam 0
4      python main.py --source 1    # another webcam
5      python main.py --keyboard    # no camera, keyboard debug controls (see inputs.py)
6      python main.py --debug       # draw collision circles
7
8  Install: pip install pygame ultralytics mediapipe opencv-python
9
10 This file only reads the command line, picks an input source (camera or
11 keyboard) and hands it to App, which runs everything else.
12 """
13 import argparse
14
15 import config
16 from app import App
17 from inputs import KeyboardInput
18
19
20 def main():
21     # Describe the command-line options; argparse also builds '--help' from this.
22     ap = argparse.ArgumentParser()
23     ap.add_argument("--source", default=config.CAMERA_SOURCE, help="webcam index or video path")
24     ap.add_argument("--model", default=config.POSE_MODEL)
25     ap.add_argument("--keyboard", action="store_true", help="play without a camera")
26     ap.add_argument("--debug", action="store_true")
27     args = ap.parse_args()
28
29     # Both input sources have the same methods (see inputs.InputSource), so the
30     # rest of the game never needs to know which one it got.
31     if args.keyboard:
32         source = KeyboardInput()
33     else:
34         # Imported here, not at the top: loading torch/ultralytics/mediapipe takes
35         # seconds and is not needed in keyboard mode.
36         from vision import VisionInput # heavy imports only in camera mode
37         source = VisionInput(args.source, args.model)
38     App(source, debug=args.debug).run()
39
40
41 # Run main() only when this file is started directly (python main.py),
42 # not when another file imports it.
43 if __name__ == "__main__":
44     main()

```

app.py (51 lines)

Owens the window and the three scenes, and switches between them. Each scene's `run()` returns where to go next.

Lines	What happens there
1-6	Docstring explaining scenes.
7-15	Imports of every scene and helper.
18-22	Constructor: start pygame, title the window.
24-31	Create the display first (images can only be converted for fast drawing once a window exists), then load maps, then the scenes.
33-51	The switchboard loop: menu -> chosen scene -> back to menu, until quit. The <code>finally</code> block always stops the camera thread and closes pygame, even after a crash.

Full source

```

1  """App: owns the single window and switches between menu, game and test scenes.
2
3  A "scene" is a screen with its own loop (menu, game, 1P test). Each scene's
4  run() keeps drawing frames until the player leaves it, then returns what
5  should happen next; App.run() is the switchboard between them.
6  """
7  import pygame
8
9  import config
10 from display import Display
11 from game import Game
12 from menu import Menu
13 from maps import load_maps
14 from test_scene import TestScene
15 from viewport import view_size
16
17
18 class App:
19     def __init__(self, input_source, debug=False):
20         pygame.init() # start pygame: video, fonts, events
21         pygame.display.set_caption(config.GAME_TITLE) # window title bar text
22         self.input = input_source
23
24         # The window must exist before images can be converted for fast drawing,
25         # so Display comes first, then the maps (which load and convert images).
26         self.display = Display(view_size(), input_source)
27         self.maps = load_maps()
28         # Every scene shares the same display (window) and input source.
29         self.menu = Menu(self.display, self.maps, input_source)
30         self.game = Game(self.display, input_source, debug)
31         self.test = TestScene(self.display, input_source)
32
33     def run(self):
34         # try/finally: even if the game crashes, the camera thread is stopped and
35         # the window is closed properly.
36         try:
37             while True:
38                 choice = self.menu.run() # blocks until the player picks something
39                 if choice is None: # Esc / window closed on the title page
40                     break
41                 if choice[0] == "test":
42                     scene = self.test.run()
43                 else:
44                     _, players, map_index = choice
45                     scene = self.game.run(players, self.maps[map_index])
46                 if scene == "quit": # window closed during a scene
47                     break
48                 # otherwise the scene returned "menu": loop back to the menu
49         finally:
50             self.input.close()
51             pygame.quit()

```

config.py (155 lines)

Every number that changes how the game looks or feels. Nothing here runs logic; the other files read these values. The Part IV reference table lists every setting.

Lines	What happens there
1-10	Docstring: units and colour conventions.
11-12	Paths: the project folder and the assets folder, built from this file's location.
15-17	Helper: full path of an asset picture.
20-32	Maps: reference height, the list of maps (name, pictures, spawn, scale) and how high the character starts.
34-47	Game view size, window layout, frame rate, physics substeps, camera smoothing, sky colour.
49-65	Character size and the geometry measured on the 500 px sprite canvas (head, joints, feet).
66-71	The four legs: hip position, side, rest pose.
72-84	Control schemes for 1-4 players: (player, limb, leg) rows.
85-88	Player limit, leg stretch for 1 player, where the shoe starts in the shin picture.
90-95	Flag size and placement settings.
97-110	Physics constants.
112-150	Camera, model files, keypoint threshold, face matching and crop settings, mouth and head-tilt thresholds, smoothing, colours.
152-155	Title and credit text.

Full source

```

1  """All tunable numbers live here so gameplay can be tweaked without touching logic.
2
3  Every other file does `import config` and reads values like `config.GRAVITY`.
4  Units: "px" means map pixels (the mountain map's pixels), angles are degrees,
5  times are seconds, colours are (R, G, B) for pygame or (B, G, R) for OpenCV.
6  """
7  import os
8
9  # Folder this file is in = the project folder. Building paths from it means the
10 # game finds its files no matter which folder you start Python from.
11 ROOT = os.path.dirname(os.path.abspath(__file__))
12 ASSET_DIR = os.path.join(ROOT, "assets")
13
14
15 def asset(name):
16     """Full path of an image in the assets folder, e.g. asset("head.png")."""
17     return os.path.join(ASSET_DIR, name)
18
19
20 # ----- maps -----
21 # Every map is scaled to MAP_HEIGHT px tall (times its "scale"), so character size,
22 # view and physics feel the same on all maps; scale 1.5 = map 1.5x as big compared to
23 # the character. spawn = (x, y) as fractions of the map: the character drops from
24 # there to the floor. The flag goes on the top-right high ground.
25 MAP_HEIGHT = 2164
26 MAPS = [
27     {"name": "Mountain", "background": "background.png", "foreground": "foreground.png",
28      "spawn": (0.06, 0.0)},
29     {"name": "Cave", "background": "background-cave.png", "foreground": "foreground-cave.png",
30      "spawn": (0.04, 0.84), "scale": 1.5},
31 ]
32 SPAWN_LIFT = 1 / 8          # start this far (MAP_HEIGHT fraction) above the floor, or mid-gap if lower
33
34 # ----- map / view -----
35 VIEW_FRACTION = 1 / 3      # game view height = 1/3 of the map height
36 VIEW_ASPECT = 4096 / 2164  # view width / height (the mountain map's shape)
37 WINDOW_SCALE = 1.0         # max scale of the window (it also shrinks to fit the screen)
38 # one window: game screen left, camera right (see display.py), sizes relative to the game view width
39 LAYOUT_MARGIN = 0.025      # gap between window edge / game / camera
40 LAYOUT_CAMERA_WIDTH = 0.40  # camera panel width
41 LAYOUT_SCREEN_FILL = 0.92   # window may use at most this fraction of the desktop
42 LAYOUT_BG = (40, 42, 48)    # dark grey around the two panels
43 START_FULLSCREEN = False    # F11 toggles fullscreen at any time
44 FPS = 60                   # frames drawn per second (the loop waits to keep this rate)
45 PHYSICS_SUBSTEPS = 3        # physics steps per rendered frame (stability)
46 CAMERA_LERP = 6.0           # how fast the game view follows the character (1/s)
47 SKY_COLOR = (135, 206, 235) # fills transparent gaps behind the map images
48
49 # ----- character size -----
50 # All character pngs share a 500x500 canvas with the same scale.
51 # character-demo.png canvas height = CHARACTER_FRACTION * map height.
52 CHARACTER_FRACTION = 1 / 8
53 SPRITE_CANVAS = 500        # size of the character png canvases
54
55 # Geometry measured on the 500x500 canvas (px)
56 HEAD_CENTER = (250, 250)
57 HEAD_RADIUS = 95           # collision circle of the head
58 LEG_PIVOT = (249.5, 190.5) # joint at the top of leg-part.png / leg-part-with-shoe.png
59 SEGMENT_LENGTH = 117       # pivot -> pivot of leg-part.png
60 FOOT_OFFSET = (20, 160)    # shoe centre from LEG_PIVOT while the sprite hangs down (toe = +x)
61 FOOT_RADIUS = 30           # collision circle of the shoe
62
63 # name: (hip offset from head centre, side (-1 left / +1 right), rest thigh, rest bend)
64 # Angles in degrees, "outward" convention: 0 = pointing down, 90 = pointing
65 # sideways away from the body, 180 = pointing up. bend = shin angle - thigh angle.
66 LEGS = {
67     "upper_left": ((-75, -55), -1, 145, 20),
68     "upper_right": ((75, -55), +1, 145, 20),
69     "lower_left": ((-85, 50), -1, 50, -50),
70     "lower_right": ((85, 50), +1, 50, -50),

```

```

71 }
72 # Who drives which leg, per number of players: (player index, body limb, character leg).
73 # Limbs are named by screen side of the mirrored camera image (= the player's own side).
74 # Players are numbered left -> right as they stand in front of the camera.
75 # The character only gets the legs listed in its scheme.
76 CONTROL_SCHEMES = {
77     1: [(0, "left_arm", "lower_left"), (0, "right_arm", "lower_right")],
78     2: [(0, "left_arm", "upper_left"), (0, "right_arm", "upper_right"),
79         (1, "left_arm", "lower_left"), (1, "right_arm", "lower_right")],
80     3: [(0, "left_arm", "upper_left"), (0, "right_arm", "upper_right"),
81         (1, "left_arm", "lower_left"), (2, "right_arm", "lower_right")],
82     4: [(0, "left_arm", "upper_left"), (1, "right_arm", "upper_right"),
83         (2, "left_arm", "lower_left"), (3, "right_arm", "lower_right")],
84 }
85 MAX_PLAYERS = 4 # highest player count offered in the menu
86 # leg length multiplier per player count (only the black sticks stretch, shoes keep their size)
87 LEG_STRETCH = {1: 1.5}
88 SHOE_TOP = 300 # canvas y where the shoe starts on leg-part-with-shoe.png
89
90 # ----- flag --
91 # flag.png stands on the highest ground in the right-most FLAG_SEARCH part of the map
92 # that has room above it for the whole flag.
93 FLAG_SEARCH = 0.1 # fraction of the map width searched for the peak
94 FLAG_FRACTION = 1 / 9 # flag.png canvas height relative to MAP_HEIGHT
95 FLAG_BASE = (160, 492) # canvas px of flag.png that sits on the ground (pole foot)
96
97 # ----- physics --
98 GRAVITY = 2000.0 # px/s^2 (map pixels)
99 MAX_SPEED = 1500.0 # px/s, clamp on body velocity
100 AIR_DRAG = 0.3 # 1/s
101 LEG_MAX_SPEED = 420.0 # deg/s, how fast a leg joint can follow the arm
102 SOLVER_ITERATIONS = 4 # passes over all contacts per physics step (more = stiffer, slower)
103 WALKABLE_SLOPE = 55.0 # deg; feet only get friction on surfaces flatter than this
104 CONTACT_EPS = 2.0 # px; a foot this close to the ground still counts as touching
105 STICKY_GRAB_DIST = 8.0 # px; a sticky foot grabs ground within this distance
106 STICKY_BREAK_DIST = 40.0 # px; a sticky foot tears off if pulled this far from its anchor
107 STICKY_STIFFNESS = 0.6 # share of a glued foot's error fixed per solver pass (1 = all)
108 LEG_BLOCK_TOLERANCE = 3.0 # px a free foot may sink into rock while another foot is glued
109 STICKY_SINK = 6.0 # px a sticky/glued foot may sink into rock while another foot is glued
110 HEAD_FRICTION = 6.0 # 1/s, damping while the head scrapes the ground
111
112 # ----- vision --
113 CAMERA_SOURCE = "0" # webcam index or video path
114 POSE_MODEL = "yolo26n-pose.pt" # YOLO "nano" pose model, downloaded automatically
115 # Use yolo26n-pose.engine (TensorRT, made for THIS GPU) when it exists and a GPU is present.
116 # Rebuild it after changing GPU/driver/TensorRT: python export_engine.py
117 USE_TENSORRT = True
118 FACE_MODEL = os.path.join(ROOT, "face_landmarker.task") # MediaPipe face model file
119 FACE_URL = ("https://storage.googleapis.com/mediapipe-models/face_landmarker/"
120            "face_landmarker/float16/1/face_landmarker.task")
121 MIRROR_CAMERA = True # flip the webcam so it behaves like a mirror
122 KEYPOINT_THR = 0.5 # keypoints with lower confidence (0..1) count as "not seen"
123
124 # face <-> pose matching (pixel closeness)
125 FACE_MATCH_FACTOR = 1.0 # max face-to-head distance = factor * shoulder width
126 FACE_MATCH_MIN_PX = 40
127 FACE_CROP_FALLBACK = True # if the full-frame face search misses a player, retry on a head crop
128 CROP_SCALE = 2.4 # head crop size = spread of nose/eyes/ears * this
129 CROP_DOWN = 0.25 # move the crop down (in spreads) so chin and mouth are inside
130 MIN_CROP = 24 # px; smaller heads are skipped (too few pixels for a face mesh)
131 UPSCALE_TO = 256 # small crops are enlarged to this before the face model
132
133 # mouth -> sticky feet
134 MOUTH_OPEN_T, MOUTH_CLOSE_T = 0.35, 0.25 # hysteresis on lip gap / lip width
135 MOUTH_EMA = 0.5 # smoothing of the mouth score (1 = no smoothing)
136 MOUTH_TIMEOUT = 0.5 # s; forget the mouth state if the face is lost for this long
137
138 # 1 player: sideways head tilt (roll) picks which foot the open mouth sticks.
139 # head tilted to the right (right ear lower) -> right-arm leg, tilted left -> left-arm leg.
140 HEAD_TILT_PLAYERS = {1} # player counts that use this

```

```

141 HEAD_TILT_ANGLE = 10.0      # deg the ear line must tilt to switch side (in between: keep last)
142 HEAD_TILT_INVERT = False    # True: swap (tilt right -> left-arm leg)
143
144 # arm angles -> legs
145 ANGLE_EMA = 0.6              # smoothing of arm angles (1 = no smoothing)
146
147 # player colours (BGR for OpenCV): red, green, blue, yellow
148 PLAYER_COLORS_BGR = [(0, 0, 255), (0, 220, 0), (255, 120, 0), (0, 220, 255)]
149 PLAYER_COLORS_RGB = [tuple(reversed(c)) for c in PLAYER_COLORS_BGR]
150 BODY_COLOR_BGR = (150, 150, 150) # grey for non-controlling body parts in the preview
151
152 # ----- menu -----
153 GAME_TITLE = "All For One"
154 GAME_CREDIT = "From The Group MeEeEe"
155 GAME_CREDIT_EMOJI = "[sheep emoji]" # sheep

```

display.py (120 lines)

The single window. Scenes draw on a fixed-size canvas; this class scales it into the window next to the camera picture and handles fullscreen.

Lines	What happens there
1-18	Docstring with the layout sketch.
19-22	Imports.
25-40	Constructor: layout size in canvas units (game + camera + margins), create the window and the canvas.
41-52	Create the window: fullscreen at screen size, or windowed at the biggest scale that fits 92% of the desktop.
54-68	Compute where the game panel and the camera panel go for a given window size.
70-71	Switch mode.
73-78	Keys every scene shares (F11). Returns True if it used the event.
80-85	Convert a mouse position from window pixels to canvas pixels.
86-98	Draw the frame: background, scaled canvas, frame, camera; then <code>flip()</code> shows it.
100-120	Turn the newest camera frame (numpy) into a pygame image only when it changed, fit it into the panel, or show 'no camera'.

Full source

```

1  """The single app window: game screen on the left, camera preview on the right.
2
3
4  +-----+
5  | +-----+ |
6  | | game screen | | +-----+ |
7  | |           | | +-----+ |
8  | |           | | +-----+ |
9  | +-----+ |
10 +-----+
11
12 Scenes draw onto `canvas` (map-view sized); present() scales it into the
13 window. Mouse positions must go through to_canvas() before hit-testing.
14 F11 (handled in handle_event) switches between window and fullscreen.
15
16 Why a canvas: scenes always draw at the same size (1365 x 721) no matter how
17 big the window is; only present() deals with the real window size.
18 """
19 import pygame
20
21 import config
22 from ui import draw_text
23
24
25 class Display:
26     def __init__(self, canvas_size, input_source):
27         self.input = input_source
28         self.canvas_size = canvas_size
29         cw, ch = canvas_size
30         # whole layout at canvas scale (1 unit = 1 canvas px)
31         self.margin = config.LAYOUT_MARGIN * cw
32         self.full_w = cw + config.LAYOUT_CAMERA_WIDTH * cw + 3 * self.margin # game + camera + 3 gaps
33         self.full_h = ch + 4 * self.margin
34         self.fullscreen = False
35         self.cam_surface = None # last camera picture, already scaled for the panel
36         self._cam_source = None # the frame cam_surface was made from
37         self._set_mode(config.START_FULLSCREEN)
38         self.canvas = pygame.Surface(canvas_size).convert()
39
40     # ----- window -----
41     def _set_mode(self, fullscreen):
42         """Create (or re-create) the window, then work out where things go in it."""
43         self.fullscreen = fullscreen
44         if fullscreen:
45             self.window = pygame.display.set_mode((0, 0), pygame.FULLSCREEN) # (0, 0) = screen size
46         else:
47             # biggest scale that fits LAYOUT_SCREEN_FILL of the desktop (and WINDOW_SCALE)
48             desk_w, desk_h = pygame.display.get_desktop_sizes()[0]
49             s = min(config.WINDOW_SCALE, desk_w * config.LAYOUT_SCREEN_FILL / self.full_w,
50                   desk_h * config.LAYOUT_SCREEN_FILL / self.full_h)
51             self.window = pygame.display.set_mode((round(self.full_w * s), round(self.full_h * s)))
52             self._layout(*self.window.get_size())
53
54     def _layout(self, w, h):
55         """Fit the layout into a w x h window, centred (letterboxed in fullscreen)."""
56         cw, ch = self.canvas_size
57         s = min(w / self.full_w, h / self.full_h) # scale that fits both width and height
58         self.scale = s
59         left = (w - self.full_w * s) / 2 # empty space on each side, if any
60         m = self.margin * s
61         self.game_rect = pygame.Rect(round(left + m), 0, round(cw * s), round(ch * s))
62         self.game_rect.centery = h // 2
63         cam_w = round(config.LAYOUT_CAMERA_WIDTH * cw * s)
64         # camera panel: right of the game, 4:3 shape (like a webcam picture), a bit above centre
65         self.cam_rect = pygame.Rect(self.game_rect.right + round(m), 0, cam_w, round(cam_w * 3 / 4))
66         self.cam_rect.centery = round(self.game_rect.centery - self.game_rect.h * 0.08)
67         self.label_font = pygame.font.SysFont("arial", max(14, cam_w // 22), bold=True)
68         self._cam_source = None # rescale the camera image for the new size
69
70     def toggle_fullscreen(self):

```

```

71         self._set_mode(not self.fullscreen)
72
73     def handle_event(self, event):
74         """Window-level keys shared by every scene. True if the event was used."""
75         if event.type == pygame.KEYDOWN and event.key == pygame.K_F11:
76             self.toggle_fullscreen()
77             return True
78         return False
79
80     def to_canvas(self, pos):
81         """Window pixel -> canvas pixel (for mouse clicks)."""
82         # undo what present() did: subtract the game panel's corner, divide by the scale
83         return ((pos[0] - self.game_rect.x) / self.scale, (pos[1] - self.game_rect.y) / self.scale)
84
85     # ----- draw -----
86     def present(self):
87         """Put the finished canvas and the camera picture in the window and show it."""
88         self.window.fill(config.LAYOUT_BG)
89         if self.game_rect.size == self.canvas.get_size():
90             self.window.blit(self.canvas, self.game_rect) # same size: plain copy (fast)
91         else:
92             self.window.blit(pygame.transform.smoothscale(self.canvas, self.game_rect.size),
93                             self.game_rect)
94         pygame.draw.rect(self.window, (0, 0, 0), self.game_rect.inflate(6, 6), 3) # black frame
95         self._draw_camera()
96         # Everything above was drawn into a hidden buffer; flip() shows it all at
97         # once, so the player never sees a half-drawn frame.
98         pygame.display.flip()
99
100    def _draw_camera(self):
101        frame, fresh = self.input.get_preview()
102        # convert numpy -> pygame image only when a NEW camera frame arrived
103        # (the camera runs slower than the game; re-converting each frame is waste)
104        if frame is not None and (fresh or self._cam_source is not frame):
105            self._cam_source = frame
106            h, w = frame.shape[:2]
107            img = pygame.image.frombuffer(frame.tobytes(), (w, h), "RGB")
108            fit = min(self.cam_rect.w / w, self.cam_rect.h / h) # fit inside the panel, keep shape
109            self.cam_surface = pygame.transform.smoothscale(img, (round(w * fit), round(h * fit)))
110        if self.cam_surface is not None and self._cam_source is not None:
111            rect = self.cam_surface.get_rect(center=self.cam_rect.center)
112            self.window.blit(self.cam_surface, rect)
113        else:
114            rect = self.cam_rect
115            pygame.draw.rect(self.window, (20, 20, 24), rect)
116            draw_text(self.window, "no camera (keyboard mode)", self.label_font,
117                      (200, 200, 200), rect.center, width=1)
118            pygame.draw.rect(self.window, (0, 0, 0), rect.inflate(6, 6), 3)
119            draw_text(self.window, "CAMERA", self.label_font, (230, 230, 230),
120                      (rect.centerx, rect.top - self.label_font.get_linesize()), width=1)

```

menu.py (144 lines)

The menu screen with its three pages. Returns what the player chose.

Lines	What happens there
1-7	Docstring: how the page loop works.
8-13	Imports.
14-19	Help text on the player-count buttons.
22-43	Constructor: backdrop picture, demo character, fonts.
45-65	Create every button: PLAY, 1P TEST, the four player-count buttons, BACK and one picture button per map.
67-103	The menu loop: handle quit/F11, convert mouse positions to canvas pixels, react to keys and clicks depending on the page, then draw.
105-113	Draw the credit line with a separate emoji font for the sheep.
115-144	Draw the current page.

Full source

```

1  """Main menu: title + Play / Test, then the player-count picker, then the map picker.
2
3  The menu is one loop with a `page` variable ("title" -> "players" -> "maps").
4  Every frame it handles clicks/keys for the current page and redraws that page.
5  All positions are fractions of the canvas size (e.g. h * 0.80 = 80% down),
6  so the layout stays right if the canvas size ever changes.
7  """
8  import pygame
9
10 import config
11 from ui import Button, MapButton, draw_text
12
13 # two lines of help text shown on each player-count button
14 PLAYER_INFO = {
15     1: ["two long legs", "left arm + right arm"],
16     2: ["P1 arms: top legs", "P2 arms: bottom legs"],
17     3: ["P1 arms: top legs", "P2 left arm, P3 right arm"],
18     4: ["one arm = one leg", "P1 P2 top, P3 P4 bottom"],
19 }
20
21
22 class Menu:
23     """run() returns ("play", players, map index), ("test", 1) or None to quit."""
24
25     def __init__(self, display, maps, input_source):
26         self.display = display
27         self.canvas = display.canvas
28         self.input = input_source
29         w, h = self.canvas.get_size()
30         # background picture: the whole first map shrunk to the canvas
31         self.backdrop = pygame.transform.smoothscale(maps[0].terrain.surface, (w, h))
32         demo = pygame.image.load(config.asset("character-demo.png")).convert_alpha()
33         size = int(h * 0.42)
34         self.demo = pygame.transform.smoothscale(demo, (size, size))
35
36         # SysFont("a,b") tries font a first, then b if a is not installed
37         self.title_font = pygame.font.SysFont("arialblack,arial", int(h * 0.13), bold=True)
38         self.big_font = pygame.font.SysFont("arialblack,arial", int(h * 0.06), bold=True)
39         self.mid_font = pygame.font.SysFont("arialblack,arial", int(h * 0.04), bold=True)
40         self.small_font = pygame.font.SysFont("arial", int(h * 0.024), bold=True)
41         self.credit_font = pygame.font.SysFont("arial", int(h * 0.034), bold=True)
42         self.emoji_font = pygame.font.SysFont("segoeuemoji", int(h * 0.04))
43         self.clock = pygame.time.Clock()
44
45         cx = w // 2
46         self.play = Button("PLAY", (cx, int(h * 0.80)), (int(w * 0.22), int(h * 0.12)),
47                             self.big_font, (220, 60, 60))
48         self.test = Button("1P TEST", (int(w * 0.80), int(h * 0.80)), (int(w * 0.15), int(h * 0.09)),
49                             self.mid_font, (60, 110, 200))
50         bw, bh = int(w * 0.21), int(h * 0.22)
51         self.counts = []
52         for i, n in enumerate(range(1, config.MAX_PLAYERS + 1)):
53             x = int(w * (0.5 + (i - 1.5) * 0.235)) # 4 buttons spread around the centre
54             label = f"{n} PLAYER" + ("S" if n > 1 else "")
55             self.counts.append((n, Button(label, (x, int(h * 0.55)), (bw, bh), self.big_font,
56                                             config.PLAYER_COLORS_RGB[i], PLAYER_INFO[n],
57                                             self.small_font)))
58         self.back = Button("BACK", (cx, int(h * 0.85)), (int(w * 0.14), int(h * 0.09)),
59                             self.big_font, (90, 90, 90))
60
61         self.maps = []
62         for i, game_map in enumerate(maps):
63             x = int(w * (0.5 + (i - (len(maps) - 1) / 2) * 0.42))
64             self.maps.append(MapButton(game_map.name.upper(), game_map.thumbnail,
65                                         (x, int(h * 0.58)), self.mid_font))
66
67         self.players = 1
68
69     def run(self):
70         page = "title"
71         while True:
72             self.clock.tick(config.FPS)

```

```

71     for event in pygame.event.get():
72         if event.type == pygame.QUIT:
73             return None
74         if self.display.handle_event(event):
75             continue
76         # mouse events have a window position: convert it to canvas pixels,
77         # because the buttons were placed on the canvas
78         pos = self.display.to_canvas(event.pos) if hasattr(event, "pos") else (-1, -1)
79         key = event.key if event.type == pygame.KEYDOWN else None
80         if key == pygame.K_ESCAPE:
81             if page == "title":
82                 return None
83             page = "players" if page == "maps" else "title"
84         if page == "title":
85             if self.play.clicked(event, pos) or key == pygame.K_RETURN:
86                 page = "players"
87             elif self.test.clicked(event, pos) or key == pygame.K_t:
88                 return "test", 1
89         elif page == "players":
90             if self.back.clicked(event, pos):
91                 page = "title"
92             for n, button in self.counts:
93                 if button.clicked(event, pos) or key == pygame.K_0 + n:
94                     self.players, page = n, "maps"
95                     break
96         else:
97             if self.back.clicked(event, pos):
98                 page = "players"
99             for i, button in enumerate(self.maps):
100                 if button.clicked(event, pos) or key == pygame.K_1 + i:
101                     return "play", self.players, i
102     self.draw(page)
103     self.display.present()
104
105 def draw_credit(self, cx, cy):
106     """'From The Group MeEeE' + a colour sheep emoji (needs the emoji font)."""
107     text = self.credit_font.render(config.GAME_CREDIT, True, (255, 255, 255))
108     sheep = self.emoji_font.render(config.GAME_CREDIT_EMOJI, True, (255, 255, 255))
109     gap = 8
110     left = cx - (text.get_width() + gap + sheep.get_width()) // 2
111     draw_text(self.canvas, config.GAME_CREDIT, self.credit_font, (255, 255, 255),
112              (left + text.get_width() // 2, cy), width=2)
113     self.canvas.blit(sheep, sheep.get_rect(midleft=(left + text.get_width() + gap, cy)))
114
115 def draw(self, page):
116     c = self.canvas
117     w, h = c.get_size()
118     mouse = self.display.to_canvas(pygame.mouse.get_pos())
119     c.blit(self.backdrop, (0, 0))
120     shade = pygame.Surface((w, h), pygame.SRCALPHA)
121     shade.fill((0, 0, 0, 90))
122     c.blit(shade, (0, 0))
123     draw_text(c, config.GAME_TITLE, self.title_font, (255, 215, 60),
124              (w // 2, int(h * 0.14)), width=5)
125     self.draw_credit(w // 2, int(h * 0.245))
126     if page == "title":
127         c.blit(self.demo, self.demo.get_rect(center=(w // 2, int(h * 0.51))))
128         self.play.draw(c, mouse)
129         self.test.draw(c, mouse)
130         draw_text(c, "F11 fullscreen", self.small_font, (230, 230, 230),
131                  (int(w * 0.92), int(h * 0.96)), width=2)
132     elif page == "players":
133         draw_text(c, "How many players?", self.big_font, (255, 255, 255),
134                  (w // 2, int(h * 0.32)))
135         for _, button in self.counts:
136             button.draw(c, mouse)
137         self.back.draw(c, mouse)
138     else:
139         players = f"{self.players} player" + ("s" if self.players > 1 else "")
140         draw_text(c, f"Choose a map ({players})", self.big_font, (255, 255, 255),
141                  (w // 2, int(h * 0.32)))
142         for button in self.maps:
143             button.draw(c, mouse)
144         self.back.draw(c, mouse)

```

ui.py (77 lines)

Reusable drawing helpers for the screens.

Lines	What happens there
1-2	Docstring.
5-18	Text with a black outline: the text is pasted 8 times in black around its position, then once in colour on top.
21-31	A rounded clickable rectangle.
33-36	True for a left click inside the button.
38-54	Hover effect, shadow, body, border, label and optional subtitle lines.
57-77	A button that shows a map picture with its name; reuses Button's click test.

Full source

```

1  """Small UI helpers: clickable buttons and outlined text."""
2  import pygame
3
4
5  def draw_text(surface, text, font, color, center, outline=(0, 0, 0), width=3):
6      """Text with a dark outline so it reads on top of the busy map art."""
7      img = font.render(text, True, color)          # True = smooth (anti-aliased) letters
8      rect = img.get_rect(center=center)
9      if outline is not None:
10         # the outline = the same text in black, pasted 8 times around the real
11         # position (left, right, up, down and the diagonals), then the text on top
12         shadow = font.render(text, True, outline)
13         for dx in range(-width, width + 1, width):
14             for dy in range(-width, width + 1, width):
15                 if dx or dy:
16                     surface.blit(shadow, rect.move(dx, dy))
17             surface.blit(img, rect)
18         return rect
19
20
21  class Button:
22      """A rounded, clickable rectangle with a label (and optional lines of small text)."""
23
24      def __init__(self, text, center, size, font, color=(70, 160, 70), subtitle=None, small_font=None):
25          self.text = text
26          self.rect = pygame.Rect(0, 0, *size)
27          self.rect.center = center
28          self.font = font
29          self.color = pygame.Color(color)
30          self.subtitle = subtitle
31          self.small_font = small_font
32
33      def clicked(self, event, pos):
34          """pos = event.pos already mapped to canvas coordinates."""
35          return (event.type == pygame.MOUSEBUTTONDOWN and event.button == 1
36                  and self.rect.collidepoint(pos))
37
38      def draw(self, surface, mouse=None):
39          hover = mouse is not None and self.rect.collidepoint(mouse) # mouse over the button?
40          # hovering: 25% lighter colour and slightly bigger, so it feels "pressable"
41          fill = self.color.lerp((255, 255, 255), 0.25) if hover else self.color
42          rect = self.rect.inflate(8, 8) if hover else self.rect
43          pygame.draw.rect(surface, (0, 0, 0), rect.move(4, 5), border_radius=18) # drop shadow
44          pygame.draw.rect(surface, fill, rect, border_radius=18)                # body
45          pygame.draw.rect(surface, (0, 0, 0), rect, 4, border_radius=18)        # 4 px black border
46          if self.subtitle and self.small_font: # subtitle = list of lines
47              draw_text(surface, self.text, self.font, (255, 255, 255),
48                        (rect.centerx, rect.centery - rect.h * 0.18))
49              line_h = self.small_font.get_linesize()
50              for i, line in enumerate(self.subtitle):
51                  draw_text(surface, line, self.small_font, (255, 255, 255),
52                            (rect.centerx, rect.centery + rect.h * 0.15 + i * line_h), width=2)
53          else:
54              draw_text(surface, self.text, self.font, (255, 255, 255), rect.center)
55
56
57  class MapButton(Button):
58      """A button showing a map picture with its name underneath."""
59
60      def __init__(self, name, image, center, font, color=(60, 60, 70)):
61          pad, label_h = 14, font.get_linesize() + 6
62          size = (image.get_width() + 2 * pad, image.get_height() + label_h + 2 * pad)
63          super().__init__(name, center, size, font, color) # reuse Button's setup and clicked()
64          self.image, self.pad = image, pad
65
66      def draw(self, surface, mouse=None):
67          hover = mouse is not None and self.rect.collidepoint(mouse)
68          fill = self.color.lerp((255, 255, 255), 0.25) if hover else self.color
69          rect = self.rect.inflate(10, 10) if hover else self.rect
70          pygame.draw.rect(surface, (0, 0, 0), rect.move(4, 5), border_radius=18)
71
72          pygame.draw.rect(surface, fill, rect, border_radius=18)
73          img_rect = self.image.get_rect(midtop=(rect.centerx, rect.top + self.pad))
74          surface.blit(self.image, img_rect)
75          pygame.draw.rect(surface, (0, 0, 0), img_rect, 2)
76          draw_text(surface, self.text, self.font, (255, 255, 255),
77                    (rect.centerx, (img_rect.bottom + rect.bottom) // 2), width=2)

```

game.py (207 lines)

The level itself: the game loop, players -> legs, physics substeps, camera, timer, flag, HUD and the finish screen.

Lines	What happens there
1-10	Docstring listing the six steps of every frame.
11-20	Imports and readable leg names for the HUD.
23-37	For each (player, limb, leg) row of the scheme: copy the limb's angles to the leg as its target and set the leg's sticky flag from the mouth (and head tilt).
40-63	Constructor: fonts, clock, sprite scale (character = 1/8 of the map height), best times.
65-66	Start position of the current map.
68-74	Create a character with only the controlled legs; sprites are cached per leg length.
76-82	Back to the start: character, view, timer.
83-123	Set up the map, view, input and character, then the frame loop: events, players, physics substeps, camera, timer, flag check, drawing.
125-132	Flag touched: stop the timer, keep the best time per (map, player count).
134-137	Method wrapper around the module function.
138-150	Draw in painter's order and present.
152-157	Coloured dot on each knee showing which player drives that leg.
159-174	Player lines and the info line.
175-178	Seconds -> mm:ss.cc.
180-194	Dark overlay with COMPLETED!, time and best time.
196-201	Timer in the top-right corner (gold when finished).
203-207	Small text with a shadow.

Full source

```

1  """Game scene: ties terrain, character, camera view and the players' input together.
2
3  Every frame of Game.run():
4      1. handle keys/window events
5      2. read the players (arm angles, mouths) from the input source
6      3. turn them into leg targets and sticky flags (apply_players)
7      4. run the physics a few small steps (substeps)
8      5. move the camera view, update the timer, check the flag
9      6. draw everything and show it (display.present)
10 """
11 import pygame
12
13 import config
14 from assets import CharacterSprites
15 from character import Character
16 from ui import draw_text
17 from viewport import Viewport, view_size
18
19 LEG_LABELS = {"upper_left": "top-left", "upper_right": "top-right",
20              "lower_left": "bottom-left", "lower_right": "bottom-right"}
21
22
23 def apply_players(character, scheme, players):
24     """Players' limb angles -> leg targets, mouths (and head tilt) -> sticky feet."""
25     # scheme rows look like (0, "left_arm", "upper_left"): player 0's left arm
26     # drives the character's upper-left leg (see config.CONTROL_SCHEMES)
27     for player, limb, leg_name in scheme:
28         if player >= len(players): # vision thread not switched over yet
29             continue
30         st = players[player]
31         leg = character.legs[leg_name]
32         pose = st.limbs.get(limb)
33         if pose is not None: # arm not seen this frame: the leg keeps its last target
34             leg.set_target(pose.thigh, pose.bend)
35         # 1 player: head tilt picks a side, only that side's foot gets sticky
36         side_ok = st.sticky_side is None or limb.startswith(st.sticky_side)
37         leg.sticky = st.mouth_open and side_ok
38
39
40 class Game:
41     """run(num_players, game_map) plays until Esc ('menu') or window close ('quit').
42     Touching the flag with the head completes the level and stops the timer."""
43
44     def __init__(self, display, input_source, debug=False):
45         self.display = display
46         self.canvas = display.canvas # draw here; display.present() shows it
47         self.map = self.terrain = self.flag = self.view = None # set when a game starts
48         self.input = input_source
49         self.debug = debug
50
51         # sprite scale: the 500 px character canvas becomes 1/8 of the map height
52         self.sprite_scale = config.MAP_HEIGHT * config.CHARACTER_FRACTION / config.SPRITE_CANVAS
53         self.sprites = {} # leg stretch -> CharacterSprites
54         self.character = None
55         self.font = pygame.font.SysFont("arial", 20, bold=True)
56         self.timer_font = pygame.font.SysFont("consolas,couriernew", 40, bold=True)
57         self.clock = pygame.time.Clock() # measures frame time and limits the FPS
58         self.scheme = config.CONTROL_SCHEMES[2]
59         self.elapsed = 0.0 # timer, seconds
60         self.finished = False
61         self.best = {} # (map name, player count) -> best time this session
62         self.big_font = pygame.font.SysFont("arialblack,arial", 72, bold=True)
63         self.mid_font = pygame.font.SysFont("arialblack,arial", 34, bold=True)
64
65     def spawn_point(self):
66         return pygame.Vector2(self.map.spawn)
67
68     def build_character(self, num_players):
69         """Only the legs that someone controls, stretched for small teams."""
70         stretch = config.LEG_STRETCH.get(num_players, 1.0)

```

```

71     if stretch not in self._sprites:          # build each sprite set once, then reuse it
72         self._sprites[stretch] = CharacterSprites(self.sprite_scale, stretch)
73     legs = [leg for _, leg in self.scheme]
74     return Character(self._sprites[stretch], self.spawn_point(), legs)
75
76     def restart(self):
77         self.character.reset(self.spawn_point())
78         self.view.snap(self.character.pos)
79         self.elapsed = 0.0
80         self.finished = False
81
82     # ----- loop -----
83     def run(self, num_players, game_map):
84         self.map, self.terrain, self.flag = game_map, game_map.terrain, game_map.flag
85         self.view = Viewport(self.terrain.width, self.terrain.height, *view_size())
86         self.input.set_num_players(num_players)
87         self.scheme = config.CONTROL_SCHEMES[num_players]
88         self.character = self.build_character(num_players)
89         self.restart()
90         self.clock.tick()          # reset the clock so the first dt is small
91         while True:
92             # tick() waits so the loop runs at most FPS times per second and returns
93             # the ms since the last frame. dt is capped at 1/20 s so a hiccup (e.g.
94             # dragging the window) can't make the physics take one giant step.
95             dt = min(self.clock.tick(config.FPS) / 1000.0, 1 / 20)
96             for event in pygame.event.get():
97                 if event.type == pygame.QUIT:
98                     return "quit"
99                 if self.display.handle_event(event):    # F11 etc.
100                     continue
101                 if event.type == pygame.KEYDOWN:
102                     if event.key == pygame.K_ESCAPE or (
103                         self.finished and event.key == pygame.K_RETURN):
104                         return "menu"
105                     if event.key == pygame.K_F5:
106                         self.restart()
107                     elif event.key == pygame.K_F1:
108                         self.debug = not self.debug
109
110             players = self.input.get_players()
111             self.apply_players(players)
112             # several small physics steps per frame: smaller steps = fewer feet
113             # passing through thin rock and a more stable solver
114             sub = dt / config.PHYSICS_SUBSTEPS
115             for _ in range(config.PHYSICS_SUBSTEPS):
116                 self.character.update(sub, self.terrain)
117             self.view.follow(self.character.pos, dt)
118             if not self.finished:
119                 self.elapsed += dt
120                 if self.flag.touches(self.character.pos, self.character.head_radius):
121                     self.complete(num_players)
122
123             self.draw(players)
124
125     def complete(self, num_players):
126         """Flag reached: stop the timer and remember the best time."""
127         self.finished = True
128         key = (self.map.name, num_players)
129         best = self.best.get(key)
130         self.new_best = best is None or self.elapsed < best
131         if self.new_best:
132             self.best[key] = self.elapsed
133
134     def apply_players(self, players):
135         apply_players(self.character, self.scheme, players)
136
137     # ----- drawing -----
138     def draw(self, players):
139         # Painter's order: things drawn later cover earlier ones.
140         rect = self.view.rect

```

```

141     offset = pygame.Vector2(rect.topleft)
142     self.canvas.blit(self.terrain.surface, (0, 0), rect) # copy just the visible part of the map
143     self.flag.draw(self.canvas, offset)
144     self.character.draw(self.canvas, offset, self.debug)
145     self.draw_leg_owners(offset)
146     self.draw_hud(players)
147     self.draw_timer()
148     if self.finished:
149         self.draw_complete()
150     self.display.present()
151
152 def draw_leg_owners(self, offset):
153     """Small dot on each knee in the colour of the player driving that leg."""
154     for player, _, leg_name in self.scheme:
155         _, knee, _ = self.character.legs[leg_name].joints(self.character.pos - offset)
156         pygame.draw.circle(self.canvas, (0, 0, 0), knee, 7) # black ring
157         pygame.draw.circle(self.canvas, config.PLAYER_COLORS_RGB[player], knee, 5)
158
159 def draw_hud(self, players):
160     """Status text in the top-left corner."""
161     y = 10
162     n = len({p for p, _, _ in self.scheme}) # number of players
163     for i in range(n):
164         st = players[i] if i < len(players) else None
165         legs = ", ".join(LEG_LABELS[leg] for p, _, leg in self.scheme if p == i)
166         state = ("STICKY" if st.mouth_open else "free") if st and st.visible else "not seen"
167         self._text(f"P{i + 1} [{legs}]: {state}", (10, y), config.PLAYER_COLORS_RGB[i])
168         y += 24
169     height = self.terrain.height - self.character.pos.y # y grows downward
170     self._text(f"{self.map.name} height {height / self.terrain.height * 100:.0f}% ",
171               f"F5 restart F1 debug F11 fullscreen Esc menu {self.clock.get_fps():.0f} FPS",
172               (10, y), (255, 255, 255))
173
174 @staticmethod
175 def format_time(t):
176     """83.456 s -> "01:23.46"."""
177     minutes, seconds = divmod(t, 60)
178     return f"{int(minutes):02d}:{seconds:05.2f}"
179
180 def draw_complete(self):
181     """The "COMPLETED!" overlay with time and best time."""
182     w, h = self.canvas.get_size()
183     shade = pygame.Surface((w, h), pygame.SRCALPHA) # see-through dark layer
184     shade.fill((0, 0, 0, 110)) # alpha 110 of 255
185     self.canvas.blit(shade, (0, 0))
186     draw_text(self.canvas, "COMPLETED!", self.big_font, (255, 215, 60), (w // 2, h * 0.34), width=5)
187     draw_text(self.canvas, f"time {self.format_time(self.elapsed)}", self.mid_font,
188               (255, 255, 255), (w // 2, h * 0.50))
189     n = len({p for p, _, _ in self.scheme})
190     best = ("NEW BEST!" if self.new_best
191            else f"best {self.format_time(self.best[(self.map.name, n)])}")
192     draw_text(self.canvas, best, self.mid_font, (120, 255, 120), (w // 2, h * 0.59))
193     draw_text(self.canvas, "Enter: menu F5: play again", self.font,
194               (255, 255, 255), (w // 2, h * 0.70), width=2)
195
196 def draw_timer(self):
197     text = self.format_time(self.elapsed)
198     color = (255, 215, 60) if self.finished else (255, 255, 255) # gold when finished
199     img = self.timer_font.render(text, True, (255, 255, 255)) # only to measure the width
200     center = (self.canvas.get_width() - img.get_width() // 2 - 20, 32) # top-right corner
201     draw_text(self.canvas, text, self.timer_font, color, center)
202
203 def _text(self, msg, pos, color):
204     """Small text with a 2 px black shadow so it reads on any background."""
205     shadow = self.font.render(msg, True, (0, 0, 0))
206     self.canvas.blit(shadow, (pos[0] + 2, pos[1] + 2))
207     self.canvas.blit(self.font.render(msg, True, color), pos)

```

test_scene.py (77 lines)

1-player test screen: same input and leg logic as the game, but no physics, so the player can check the controls.

Lines	What happens there
1-2	Docstring.
3-9	Imports.
11-13	Settings: one player, bigger character, white background.
16-31	Constructor: centre position, scheme, its own (bigger) character and fonts.
33-53	Loop: events, players, <code>apply_players</code> , move only the leg angles, draw.
55-77	White background, character, the mouth state in huge letters, sticky side and head roll, help line.

Full source

```

1  """1-player test: the character hangs fixed in front of a white background so a
2  player can check arm -> leg control, mouth (sticky) and head tilt without physics."""
3  import pygame
4
5  import config
6  from assets import CharacterSprites
7  from character import Character
8  from game import apply_players
9  from ui import draw_text
10
11  PLAYERS = 1
12  TEST_CHARACTER_SCALE = 0.62  # sprite scale (canvas px per png px), bigger than in game
13  BG = (255, 255, 255)
14
15
16  class TestScene:
17      """run() until Esc ('menu') or window close ('quit')."""
18
19      def __init__(self, display, input_source):
20          self.display = display
21          self.canvas = display.canvas
22          self.input = input_source
23          w, h = self.canvas.get_size()
24          self.center = pygame.Vector2(w / 2, h * 0.56)
25          self.scheme = config.CONTROL_SCHEMES[PLAYERS]
26          sprites = CharacterSprites(TEST_CHARACTER_SCALE, config.LEG_STRETCH.get(PLAYERS, 1.0))
27          self.character = Character(sprites, self.center, [leg for _, leg in self.scheme])
28          self.huge_font = pygame.font.SysFont("arialblack,arial", int(h * 0.12), bold=True)
29          self.mid_font = pygame.font.SysFont("arialblack,arial", int(h * 0.05), bold=True)
30          self.small_font = pygame.font.SysFont("arial", int(h * 0.03), bold=True)
31          self.clock = pygame.time.Clock()
32
33      def run(self):
34          self.input.set_num_players(PLAYERS)
35          self.character.reset(self.center)
36          self.clock.tick()
37          while True:
38              dt = min(self.clock.tick(config.FPS) / 1000.0, 1 / 20)
39              for event in pygame.event.get():
40                  if event.type == pygame.QUIT:
41                      return "quit"
42                  if self.display.handle_event(event):
43                      continue
44                  if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
45                      return "menu"
46              players = self.input.get_players()
47              apply_players(self.character, self.scheme, players)
48              # no Character.update() here = no physics: only the leg angles are
49              # moved towards the arm angles, the body never falls or moves
50              for leg in self.character.legs.values():  # legs move, body stays put
51                  leg.update_angles(dt)
52              self.draw(players[0] if players else None)
53              self.display.present()
54
55      def draw(self, st):
56          c = self.canvas
57          w, h = c.get_size()
58          c.fill(BG)
59          self.character.draw(c, pygame.Vector2(0, 0))  # offset 0: the position is already a canvas position
60
61          if st is None or not st.visible:
62              mouth, color = "NOT SEEN", (150, 150, 150)
63          elif st.mouth_open:
64              mouth, color = "MOUTH OPEN", (40, 200, 40)
65          else:
66              mouth, color = "MOUTH CLOSED", (220, 60, 60)
67          draw_text(c, mouth, self.huge_font, color, (w // 2, int(h * 0.11)), width=4)
68
69          if st is not None and st.sticky_side is not None:
70              side = st.sticky_side.upper()
71
72              draw_text(c, f"sticky foot: {side}", self.mid_font, (255, 255, 255),
73                      (w // 2, int(h * 0.22)), width=3)
74              draw_text(c, f"head roll {st.head_tilt:+.0f} deg (switch at "
75                      f"+{config.HEAD_TILT_ANGLE:.0f})", self.small_font, (60, 60, 60),
76                      (w // 2, int(h * 0.285)), outline=None)
77              draw_text(c, "1 PLAYER TEST  Esc: menu  F11: fullscreen", self.small_font, (90, 90, 90),
78                      (int(w * 0.2), int(h * 0.97)), outline=None)

```

maps.py (59 lines)

Turns an entry of `config.MAPS` into a playable map.

Lines	What happens there
1-6	Docstring and imports.
8	Height of the menu picture.
11-25	Load both pictures and scale them to the common height (times the map's own scale).
26-28	Build collision, place the flag, find the start point.
30-38	Make the menu thumbnail with the flag pasted on it.
40-55	From the configured point: go up out of rock if needed, then find floor and ceiling and start above the floor (or mid-gap in a low tunnel).
58-59	Build every map in the list.

Full source

```

1  """Playable maps: terrain + flag + start point, built from the entries in config.MAPS."""
2  import pygame
3
4  import config
5  from flag import Flag
6  from terrain import Terrain
7
8  THUMB_HEIGHT = 230  # menu picture height (canvas px)
9
10
11 class GameMap:
12     def __init__(self, spec):
13         # spec = one dictionary from config.MAPS (name, image files, spawn, scale)
14         self.name = spec["name"]
15         bg = pygame.image.load(config.asset(spec["background"])).convert_alpha()
16         fg = pygame.image.load(config.asset(spec["foreground"])).convert_alpha()
17         # Resize so the map is MAP_HEIGHT tall (times its own "scale"). The
18         # character, flag and view are sized from MAP_HEIGHT, so they look the
19         # same on every map; "scale" makes a map bigger compared to them.
20         height = round(config.MAP_HEIGHT * spec.get("scale", 1.0))  # MAP_HEIGHT x map scale
21         scale = height / fg.get_height()
22         if abs(scale - 1) > 1e-3:  # skip if already right
23             size = (round(fg.get_width() * scale), height)  # keep the shape
24             bg = pygame.transform.smoothscale(bg, size)
25             fg = pygame.transform.smoothscale(fg, size)
26         self.terrain = Terrain(bg, fg)
27         self.flag = Flag(self.terrain)
28         self.spawn = self._find_spawn(*spec["spawn"])
29
30         # Small picture for the map-selection menu: the whole map shrunk to
31         # THUMB_HEIGHT tall, with a shrunk flag pasted at the same relative spot.
32         t = self.terrain.surface
33         thumb = (round(t.get_width() * THUMB_HEIGHT / t.get_height()), THUMB_HEIGHT)
34         self.thumbnail = pygame.transform.smoothscale(t, thumb)
35         self.thumbnail.blit(pygame.transform.smoothscale(
36             self.flag.image, [max(1, round(v * THUMB_HEIGHT / t.get_height()))
37                             for v in self.flag.image.get_size()]
38             ),
39             self.flag.pos * THUMB_HEIGHT / t.get_height())
40
41     def _find_spawn(self, fx, fy):
42         """Drop from (fx, fy) (map fractions) to the floor below; start a little above it,
43         but not higher than the middle of the gap (so it fits in tunnels)."""
44         t = self.terrain
45         x, y = fx * t.width, fy * t.height
46         while y > 0 and t.distance(x, y) <= 0:  # start point in rock: go up into air
47             y -= 2
48         floor = y
49         while floor < t.height - 1 and t.distance(x, floor) > 0:  # walk down to the floor
50             floor += 2
51         ceiling = y
52         while ceiling > 0 and t.distance(x, ceiling) > 0:  # walk up to the ceiling (or top)
53             ceiling -= 2
54         lift = config.MAP_HEIGHT * config.SPAWN_LIFT
55         # normally start `lift` above the floor; in a low tunnel start at its middle
56         return pygame.Vector2(x, max(floor - lift, (floor + ceiling) / 2))
57
58     def load_maps():
59         return [GameMap(spec) for spec in config.MAPS]

```

terrain.py (96 lines)

Map picture and collision via a signed distance field (see chapter 7).

Lines	What happens there
1-10	Docstring explaining the signed distance field.
11-17	Imports.
20-33	Constructor: compose the map picture (sky, background, foreground).
35-43	Build the solid mask from the foreground alpha, add invisible side walls, compute the distance field.
44-55	Two distance transforms (air -> rock, rock -> air) subtracted.
56-75	Smooth lookup of the distance with special rules outside the map.
77-85	Direction out of the rock from the distance gradient.
87-96	First rock below the open sky in a column (used for the flag).

Full source

```

1  """Map image + collision. The opaque pixels of foreground.png are solid ground.
2
3  Collision uses a "signed distance field" (SDF): a big table with one number per
4  map pixel, saying how far that pixel is from the nearest rock edge:
5      > 0 in the air (e.g. +20 = 20 px away from the nearest rock)
6      < 0 inside rock (e.g. -5 = 5 px deep inside)
7      = 0 exactly on the surface
8  With it, "is this foot inside the ground, how deep, and which way is out?"
9  is just a table lookup, no matter how wiggly the rock outline is.
10 """
11 import math
12
13 import cv2
14 import numpy as np
15 import pygame
16
17 import config
18
19
20 class Terrain:
21     """Holds the rendered map and a signed distance field for collisions.
22
23     distance(p) > 0 in the air, < 0 inside rock; normal(p) points out of the rock.
24     """
25
26     def __init__(self, background, foreground):
27         self.width, self.height = foreground.get_size()
28         # One flat picture of the whole map (sky colour, then background, then
29         # foreground on top). The game just copies a rectangle of it each frame.
30         self.surface = pygame.Surface((self.width, self.height)).convert()
31         self.surface.fill(config.SKY_COLOR)
32         self.surface.blit(background, (0, 0))
33         self.surface.blit(foreground, (0, 0))
34
35         # Alpha channel of the foreground = how opaque each pixel is (0..255).
36         # array_alpha gives it as [x][y]; .T flips it to [y][x] (row, column),
37         # which is how numpy/OpenCV index images. More than half opaque = rock.
38         solid = pygame.surfarray.array_alpha(foreground).T > 127 # (h, w)
39         solid[:, :2] = True # invisible walls on the map's left/right edges
40         solid[:, -2:] = True
41         self.sdf = self._signed_distance(solid)
42
43     @staticmethod
44     def _signed_distance(solid):
45         # cv2.distanceTransform: for every non-zero pixel, distance to the nearest
46         # zero pixel. Run it twice: once on the air (distance to rock) and once on
47         # the rock (distance to air); air distance minus rock distance gives one
48         # signed number per pixel.
49         air = (~solid).astype(np.uint8)
50         rock = solid.astype(np.uint8)
51         out = cv2.distanceTransform(air, cv2.DIST_L2, 5) # air px -> distance to rock
52         inside = cv2.distanceTransform(rock, cv2.DIST_L2, 5)
53         return (out - inside).astype(np.float32)
54
55     # ----- queries -----
56     def distance(self, x, y):
57         """Bilinear sample of the SDF. Above the map the top row continues upward:
58         where it is sky, open sky; where it is rock (side walls, a solid top
59         border like the cave's), rock that gets deeper going up, so anything
60         poking through is pushed back down into the map instead of escaping."""
61         outside = max(0.0, -x, x - (self.width - 1.001)) # past the side walls = deeper in rock
62         above = max(0.0, -y) # how far above the map's top edge
63         # clamp the point into the table so we never index outside it
64         x = min(max(x, 0.0), self.width - 1.001)
65         y = min(max(y, 0.0), self.height - 1.001)
66         # Bilinear interpolation: the point usually lies between 4 table cells;
67         # blend them by how close the point is to each (fx, fy = 0.1 fractions).
68         # This makes the distance change smoothly instead of in 1 px steps.
69         x0, y0 = int(x), int(y)
70         fx, fy = x - x0, y - y0

```

```

71     s = self.sdf
72     top = s[y0, x0] * (1 - fx) + s[y0, x0 + 1] * fx
73     bot = s[y0 + 1, x0] * (1 - fx) + s[y0 + 1, x0 + 1] * fx
74     d = float(top * (1 - fy) + bot * fy) - outside
75     return d - above if d < 0 else d      # rock continues (deeper) above the map; sky stays sky
76
77     def normal(self, x, y, h=2.0):
78         """Unit vector pointing out of the rock = direction the distance grows fastest.
79         Measured by sampling the distance h px left/right and up/down (the gradient)."""
80         dx = self.distance(x + h, y) - self.distance(x - h, y)
81         dy = self.distance(x, y + h) - self.distance(x, y - h)
82         n = math.hypot(dx, dy)          # length of (dx, dy)
83         if n < 1e-6:                    # flat spot: no direction, guess "up"
84             return pygame.Vector2(0, -1)
85         return pygame.Vector2(dx / n, dy / n) # divide by length -> length 1
86
87     def ground_y(self, x):
88         """Top of the ground under the open sky in column x: the first rock below the
89         first air from the top (skips a solid border along the map's top edge).
90         0 if the column is rock from the top down (no sky)."""
91         col = self.sdf[:, int(min(max(x, 0), self.width - 1))] # one column of the table
92         air = np.nonzero(col > 0)[0] # row numbers that are air
93         if not len(air) or air[0] > self.height * 0.02:        # no sky near the top
94             return 0
95         rock = np.nonzero(col[air[0]:] < 0)[0] # rock rows below that air
96         return int(air[0] + rock[0]) if len(rock) else self.height

```

flag.py (47 lines)

The goal flag.

Lines	What happens there
1-4	Docstring and imports.
7-17	Load and size the flag; build its pixel mask.
19-32	Find the right-most ground with room for the flag and take the highest point near it; place the pole foot there.
34-43	Pixel-perfect test: draw the head as a circle mask and check overlap with the flag mask.
45-47	Draw at its screen position.

Full source

```

1  """Goal flag: stands on the top-right peak; touching it with the head wins."""
2  import pygame
3
4  import config
5
6
7  class Flag:
8      def __init__(self, terrain):
9          img = pygame.image.load(config.asset("flag.png")).convert_alpha()
10         # Size: FLAG_FRACTION of the reference map height, so the flag is the same
11         # size on every map no matter how big that map is.
12         scale = config.MAP_HEIGHT * config.FLAG_FRACTION / img.get_height() # same size on every map
13         self.image = pygame.transform.smoothscale(
14             img, (round(img.get_width() * scale), round(img.get_height() * scale)))
15         # A mask is a yes/no grid of which pixels are visible; used for exact
16         # pixel-perfect touch detection (not just the rectangle around the flag).
17         self.mask = pygame.mask.from_surface(self.image)
18
19         # stand on the highest ground near the right edge, fully inside the map:
20         # find the right-most column with room above it for the whole flag, then
21         # take the highest such ground within FLAG_SEARCH of the map width from there
22         base_x, base_y = config.FLAG_BASE[0] * scale, config.FLAG_BASE[1] * scale # pole foot in the image
23         x_max = int(terrain.width - (self.image.get_width() - base_x)) # cloth must not leave the map
24         # columns (right to left) whose ground is low enough that the flag fits above it
25         fits = [x for x in range(x_max, int(base_x), -1) if terrain.ground_y(x) >= base_y]
26         if not fits:
27             fits = [x_max]
28         window = [x for x in fits if x >= fits[0] - terrain.width * config.FLAG_SEARCH]
29         peak_x = min(window, key=terrain.ground_y) # smallest y = highest ground on screen
30         # place the image so its pole foot (base_x, base_y) lands on the ground at peak_x
31         self.pos = pygame.Vector2(peak_x - base_x, terrain.ground_y(peak_x) - base_y)
32         self.rect = self.image.get_rect(topleft=self.pos)
33
34     def touches(self, center, radius):
35         """True if a circle (the head) overlaps the flag's visible pixels."""
36         r = int(radius)
37         # draw the head as a filled circle on a small transparent image ...
38         circle = pygame.Surface((2 * r + 1, 2 * r + 1), pygame.SRCALPHA)
39         pygame.draw.circle(circle, (255, 255, 255), (r, r), r)
40         # ... and ask pygame if its mask overlaps the flag mask at that position
41         # (offset = where the circle image's corner sits relative to the flag's corner)
42         offset = (int(center[0] - r - self.rect.x), int(center[1] - r - self.rect.y))
43         return self.mask.overlap(pygame.mask.from_surface(circle), offset) is not None
44
45     def draw(self, target, offset):
46         # offset = top-left of the visible view; map position - offset = screen position
47         target.blit(self.image, self.pos - offset)

```

viewport.py (44 lines)

The game camera (the visible part of the map).

Lines	What happens there
1-9	Docstring.
12-15	View size: 1/3 of the reference height, mountain map's shape.
18-22	Holds the map size, view size and top-left position.
24-28	Keep the view inside the map.
30-32	Jump to a target.
34-41	Glide towards a target (fast when far, slow when close).
42-44	Visible area as a rectangle.

Full source

```

1  """Game camera: a window onto the map that smoothly follows the character.
2
3  The map is much bigger than the screen; the viewport is the rectangle of the
4  map that is currently shown. Its top-left corner `pos` is what everything is
5  drawn relative to (screen position = map position - pos).
6  """
7  import pygame
8
9  import config
10
11
12  def view_size():
13      """Size of the game view in map pixels (the same for every map, see config.MAP_HEIGHT)."""
14      h = round(config.MAP_HEIGHT * config.VIEW_FRACTION)    # 2164 / 3 = 721
15      return round(h * config.VIEW_ASPECT), h                # 721 * (4096 / 2164) = 1365
16
17
18  class Viewport:
19      def __init__(self, map_w, map_h, w, h):
20          self.map_w, self.map_h = map_w, map_h
21          self.w, self.h = w, h
22          self.pos = pygame.Vector2()    # top-left corner in map pixels
23
24      def _clamped(self, p):
25          """Keep the view inside the map (centred if the map is smaller than the view)."""
26          def clamp(v, size, view):
27              return (size - view) / 2 if size <= view else min(max(v, 0), size - view)
28          return pygame.Vector2(clamp(p.x, self.map_w, self.w), clamp(p.y, self.map_h, self.h))
29
30      def snap(self, target):
31          """Jump straight to `target` (centred), e.g. at a restart."""
32          self.pos = self._clamped(target - pygame.Vector2(self.w, self.h) / 2)
33
34      def follow(self, target, dt):
35          """Glide towards `target`: each frame cover a fraction (CAMERA_LERP * dt) of
36          the remaining distance, so the camera moves fast when far and slows down
37          as it arrives (smooth, no jitter)."""
38          goal = self._clamped(target - pygame.Vector2(self.w, self.h) / 2)
39          self.pos += (goal - self.pos) * min(1.0, config.CAMERA_LERP * dt)
40
41      @property
42      def rect(self):
43          """The visible part of the map as a pygame.Rect (whole pixels)."""
44          return pygame.Rect(int(self.pos.x), int(self.pos.y), self.w, self.h)

```

character.py (276 lines)

The heart of the game: legs, kinematics, the physics step and drawing (see chapter 8).

Lines	What happens there
1-13	Docstring: how the physics works.
14-20	Imports and the short vector name V .
23-31	Angle helpers: wrap to -180..180 and turn towards a target the short way.
34-54	One leg: geometry (scaled and stretched), rest pose, current and target angles, sticky state.
55-56	Store the angles the player asks for.
58-63	Turn towards the target at most LEG_MAX_SPEED.
65-71	Outward angles -> screen rotations.
72-83	Hip, knee and foot positions (kinematics).
86-96	The character: head radius and the controlled legs.
98-109	Back to the start pose.
110-135	One physics step with the 'undo and hold the leg' check.
137-175	The position-based physics step: glue, predict, solve, cap, velocity.
177-190	Save / restore everything a step changes.
192-194	How deep a foot is in the rock.

Lines	What happens there
196-204	Feet deeper than allowed and deeper than before.
206-230	Glued: pull towards the anchor. Free: push out of rock, then static friction on walkable ground.
232-237	Limit a glued foot's depth.
239-244	Push the head circle out of the rock.
246-259	Contact flags, tear off over-stretched glue, forget grips of lifted feet.
260-276	Legs then head; shoe colour by state; optional debug circles.

Full source

```

1  """The four-legged character: leg kinematics, physics and drawing.
2
3  Physics is position based: every step the body falls, then contacts push the
4  body out of the rock. Because legs are rigid, a foot that is pressed into the
5  ground by its leg pushes the body away -> extending a grounded leg fast jumps.
6  Grounded feet on walkable slopes don't slide (static friction), so sweeping a
7  planted leg moves the body like walking. Sticky feet are anchored both ways,
8  so they can pull the body too (climb walls, hang from ledges).
9
10 Only the body (head) has a position and velocity. Legs have no mass: each is
11 just two angles (thigh, bend) that follow the player's arm. Where the feet are
12 follows from the body position plus those angles ("kinematics").
13 """
14 import math
15
16 import pygame
17
18 import config
19
20 V = pygame.Vector2  # short name: V(x, y) is a 2D vector with +, -, *, rotate, length...
21
22
23 def wrap_deg(a):
24     """Bring any angle into -180..180 (e.g. 270 -> -90), so 'the shortest way round' works."""
25     return (a + 180.0) % 360.0 - 180.0
26
27
28 def approach_angle(current, target, max_step):
29     """Move `current` towards `target` by at most `max_step` degrees, the short way round."""
30     diff = wrap_deg(target - current)
31     return current + max(-max_step, min(max_step, diff))
32
33
34 class Leg:
35     def __init__(self, name, hip, side, thigh, bend, scale, stretch=1.0):
36         self.name = name
37         self.side = side  # -1 left, +1 right
38         self.hip = V(hip) * scale  # offset from body centre
39         self.length = config.SEGMENT_LENGTH * stretch * scale  # hip -> knee distance
40         # the shin stick stretches, the shoe below it keeps its size (see assets.stretch_leg)
41         extra = (config.SHOE_TOP - config.LEG_PIVOT[1]) * (stretch - 1)
42         # knee -> shoe centre while the shin hangs straight down; x mirrored for left legs
43         self.foot_offset = V(side * config.FOOT_OFFSET[0], config.FOOT_OFFSET[1] + extra) * scale
44         self.foot_radius = config.FOOT_RADIUS * scale
45         self.rest = (thigh, bend)  # pose used at (re)start
46         self.thigh, self.bend = thigh, bend  # current angles (outward convention)
47         self.target_thigh, self.target_bend = thigh, bend  # angles the player asks for
48
49         self.sticky = False  # player wants this foot sticky (mouth open)
50         self.anchor = None  # world point a sticky foot is glued to
51         self.grip = None  # world point a grounded foot grips by friction
52         self.contact = False  # foot touching the ground this step
53
54     # ----- angles -----
55     def set_target(self, thigh, bend):
56         self.target_thigh, self.target_bend = thigh, bend
57
58     def update_angles(self, dt):
59         """Turn towards the target, at most LEG_MAX_SPEED deg/s. The limit smooths
60         out jerky arm tracking and caps how hard a leg can kick."""
61         step = config.LEG_MAX_SPEED * dt
62         self.thigh = wrap_deg(approach_angle(self.thigh, self.target_thigh, step))
63         self.bend = wrap_deg(approach_angle(self.bend, self.target_bend, step))
64
65     def screen_angles(self):
66         """Rotation (deg, pygame convention) of thigh and shin sprites."""
67         # "outward" angles are mirrored for left legs: multiplying by side (-1 / +1)
68         # turns them into one screen rotation. shin direction = thigh + bend.
69         return self.side * self.thigh, self.side * (self.thigh + self.bend)
70

```

```

71 # ----- kinematics -----
72 def joints(self, body):
73     """World positions of hip, knee and the foot (shoe) centre."""
74     a_thigh, a_shin = self.screen_angles()
75     hip = body + self.hip
76     # V(0, length) is the thigh hanging straight down; rotating it gives the
77     # thigh's real direction. Same for the shin, starting at the knee.
78     knee = hip + V(0, self.length).rotate(-a_thigh)
79     foot = knee + self.foot_offset.rotate(-a_shin)
80     return hip, knee, foot
81
82 def foot(self, body):
83     return self.joints(body)[2]
84
85
86 class Character:
87     def __init__(self, sprites, spawn, leg_names=tuple(config.LEGS)):
88         self.sprites = sprites
89         s = sprites.scale
90         self.head_radius = config.HEAD_RADIUS * s
91         # only the legs someone controls (1 player = 2 legs, otherwise 4)
92         self.legs = {}
93         for name in leg_names:
94             hip, side, thigh, bend = config.LEGS[name]
95             self.legs[name] = Leg(name, hip, side, thigh, bend, s, sprites.leg_stretch)
96         self.reset(spawn)
97
98     def reset(self, spawn):
99         """Back to the start: standing still at `spawn`, legs in their rest pose."""
100         self.pos = V(spawn) # body (head) centre in map pixels
101         self.vel = V() # px per second
102         self.head_contact = False
103         for leg in self.legs.values():
104             leg.thigh, leg.bend = leg.rest
105             leg.set_target(*leg.rest)
106             leg.anchor = leg.grip = None
107             leg.contact = False
108
109 # ----- physics -----
110 def update(self, dt, terrain):
111     """One physics step. While a foot is glued the body is pinned and can't
112     always get out of the way, so a leg could be forced into the rock. Then
113     the step is redone with that leg held still: it stops at the rock.
114     Feet may only sink in a little: free feet LEG_BLOCK_TOLERANCE, sticky
115     feet STICKY_SINK (looks like gripping). Glued feet pushed in by another
116     leg are kept within STICKY_SINK by the solver (_cap_sink)."""
117     before = self._snapshot() # to undo this step if needed
118     depth0 = {leg.name: self._foot_depth(leg, terrain) for leg in self.legs.values()}
119     old = {leg.name: (leg.thigh, leg.bend) for leg in self.legs.values()}
120     for leg in self.legs.values():
121         leg.update_angles(dt)
122     new = {leg.name: (leg.thigh, leg.bend) for leg in self.legs.values()}
123     self._step(dt, terrain)
124
125     # Did a moving leg drive its own foot into the rock while a foot is glued?
126     sunk = self._sunk_feet(depth0, terrain)
127     if not sunk or not any(leg.anchor is not None for leg in self.legs.values()):
128         return
129     hold = [leg for leg in sunk if new[leg.name] != old[leg.name]]
130     if hold:
131         # undo the step and redo it with those legs not moving: they stop at the rock
132         self._restore(before)
133         for leg in hold:
134             leg.thigh, leg.bend = old[leg.name]
135         self._step(dt, terrain)
136
137     def _step(self, dt, terrain):
138         # 1) sticky feet: glue to the rock when close enough; let go when not sticky
139         for leg in self.legs.values():
140             if not leg.sticky:

```

```

141         leg.anchor = None
142     elif leg.anchor is None:
143         foot = leg.foot(self.pos)
144         if terrain.distance(*foot) < leg.foot_radius + config.STICKY_GRAB_DIST:
145             leg.anchor = foot
146
147     # 2) predict: gravity speeds the body up, drag slows it a little, then move
148     start = V(self.pos)
149     self.vel.y += config.GRAVITY * dt                # +y is down on screen
150     self.vel *= max(0.0, 1.0 - config.AIR_DRAG * dt)
151     self.pos += self.vel * dt
152
153     # 3) solve: fix contacts by moving the body. Each fix can disturb another,
154     #         so go round all of them a few times (Gauss-Seidel iterations).
155     for _ in range(config.SOLVER_ITERATIONS):
156         for leg in self.legs.values():
157             self._solve_foot(leg, terrain)
158             self._solve_head(terrain)
159     for _ in range(2):                                # last word goes to the rock: no glued foot deeper than allowed
160         for leg in self.legs.values():
161             if leg.anchor is not None:
162                 self._cap_sink(leg, terrain)
163
164     # 4) velocity = how far the body really moved this step / time.
165     #         This is what turns a fast leg push into a jump: the push moved the
166     #         body, so it now has speed and keeps flying after the foot leaves.
167     self._update_contacts(terrain)
168     self.vel = (self.pos - start) / dt
169     if self.head_contact and not any(l.contact for l in self.legs.values()):
170         self.vel *= max(0.0, 1.0 - config.HEAD_FRICTION * dt) # head sliding on rock slows down
171     if self.vel.length() > config.MAX_SPEED:
172         self.vel.scale_to_length(config.MAX_SPEED)
173     # stay inside the map horizontally and not below its bottom
174     self.pos.x = min(max(self.pos.x, 0), terrain.width)
175     self.pos.y = min(self.pos.y, terrain.height)
176
177     def _snapshot(self):
178         """Copy of everything _step changes, so a step can be undone."""
179         legs = {leg.name: (V(leg.anchor) if leg.anchor is not None else None,
180                             V(leg.grip) if leg.grip is not None else None,
181                             leg.contact) for leg in self.legs.values()}
182         return V(self.pos), V(self.vel), self.head_contact, legs
183
184     def _restore(self, snap):
185         pos, vel, self.head_contact, legs = snap
186         self.pos, self.vel = V(pos), V(vel)
187         for leg in self.legs.values():
188             anchor, grip, leg.contact = legs[leg.name]
189             leg.anchor = V(anchor) if anchor is not None else None
190             leg.grip = V(grip) if grip is not None else None
191
192     def _foot_depth(self, leg, terrain):
193         """How far the foot circle is inside the rock (<= 0: outside)."""
194         return leg.foot_radius - terrain.distance(*leg.foot(self.pos))
195
196     def _sunk_feet(self, depth0, terrain):
197         """Feet deeper in the rock than allowed and deeper than before this step."""
198         out = []
199         for leg in self.legs.values():
200             limit = config.STICKY_SINK if leg.sticky else config.LEG_BLOCK_TOLERANCE
201             d = self._foot_depth(leg, terrain)
202             if d > max(limit, depth0[leg.name]) + 0.05: # no creeping in, step by step
203                 out.append(leg)
204         return out
205
206     def _solve_foot(self, leg, terrain):
207         foot = leg.foot(self.pos)
208         if leg.anchor is not None:                                # sticky: pull or push
209             # move the body part of the way so the foot gets back to its glue point
210             self.pos += (leg.anchor - foot) * config.STICKY_STIFFNESS

```

```

211         self._cap_sink(leg, terrain)                    # ...but never deeper than STICKY_SINK
212         return
213     d = terrain.distance(*foot)                          # distance of the shoe centre to rock
214     if d >= leg.foot_radius:                             # shoe circle fully in the air: nothing to do
215         return
216     n = terrain.normal(*foot)
217     self.pos += n * (leg.foot_radius - d)                # push out of the rock
218     # Friction only on walkable ground: -n.y is how much the surface faces up
219     # (1 = flat floor, 0 = vertical wall). Steeper than WALKABLE_SLOPE: slide.
220     if -n.y < math.cos(math.radians(config.WALKABLE_SLOPE)):
221         leg.grip = None                                  # too steep, slide
222         return
223     foot = leg.foot(self.pos)
224     if leg.grip is None:                                  # first touch: remember where
225         leg.grip = V(foot)
226     # tangent = along the surface. Move the body so the foot does not slide
227     # along the surface away from its grip point. If the leg sweeps backwards,
228     # the foot stays put and the BODY moves forward instead = walking.
229     tangent = V(-n.y, n.x)
230     self.pos += tangent * (leg.grip - foot).dot(tangent) # static friction
231
232 def _cap_sink(self, leg, terrain):
233     """Push the body so this glued foot is at most STICKY_SINK inside the rock."""
234     foot = leg.foot(self.pos)
235     excess = leg.foot_radius - config.STICKY_SINK - terrain.distance(*foot)
236     if excess > 0:
237         self.pos += terrain.normal(*foot) * excess
238
239 def _solve_head(self, terrain):
240     """The head is a circle too: push it out of the rock like a foot (no friction)."""
241     d = terrain.distance(*self.pos)
242     self.head_contact = d < self.head_radius + config.CONTACT_EPS
243     if d < self.head_radius:
244         self.pos += terrain.normal(*self.pos) * (self.head_radius - d)
245
246 def _update_contacts(self, terrain):
247     """After solving: which feet touch the ground, and tear off over-stretched glue."""
248     for leg in self.legs.values():
249         foot = leg.foot(self.pos)
250         if leg.anchor is not None:
251             leg.contact = True
252             if foot.distance_to(leg.anchor) > config.STICKY_BREAK_DIST:
253                 leg.anchor = None                    # torn off
254             continue
255         leg.contact = terrain.distance(*foot) < leg.foot_radius + config.CONTACT_EPS
256         if not leg.contact:
257             leg.grip = None                            # lifted: forget the friction point
258
259 # ----- drawing -----
260 def draw(self, target, offset, debug=False):
261     body = self.pos - offset                            # map position -> screen position
262     sp = self.sprites
263     for leg in self.legs.values():                      # legs first, so the head covers the hips
264         hip, knee, foot = leg.joints(body)
265         a_thigh, a_shin = leg.screen_angles()
266         sp.thigh[leg.side].draw(target, hip, a_thigh)
267         # shoe colour: purple = glued to rock, green = sticky in the air, red = normal
268         shin = (sp.shin_stuck if leg.anchor is not None
269                else sp.shin_sticky if leg.sticky else sp.shin)
270         shin[leg.side].draw(target, knee, a_shin)
271         if debug: # F1: collision circles (green glued, yellow touching, red in air)
272             color = (0, 255, 0) if leg.anchor else (255, 255, 0) if leg.contact else (255, 0, 0)
273             pygame.draw.circle(target, color, foot, leg.foot_radius, 1)
274     sp.head.draw(target, body, 0)
275     if debug:
276         pygame.draw.circle(target, (255, 0, 255), body, self.head_radius, 1)

```

assets.py (110 lines)

Prepares the character pictures (see chapter 10).

Lines	What happens there
1-9	Docstring and imports.
12-17	An image plus the joint it rotates around.
19-29	Rotate around the joint (see chapter 10).
31-35	Mirror image with mirrored joint.
38-47	Crop the empty canvas away, scale, move the joint along.
50-64	Make a leg picture longer without stretching the shoe.
67-82	Turn the red shoe green or purple while keeping its shading.
85-110	Build all pictures once: head, thigh, shin in red/green/purple, each for left and right.

Full source

```

1  """Character images: scaled, cropped, mirrored and tinted copies of the part pngs.
2
3  Everything here is done once when a character is built, so the game loop only
4  has to rotate and paste ready-made images.
5  """
6  import numpy as np
7  import pygame
8
9  import config
10
11
12  class PivotSprite:
13      """An image that is drawn rotated around a pivot point (e.g. a joint)."""
14
15      def __init__(self, image, pivot):
16          self.image = image
17          self.pivot = pygame.Vector2(pivot)  # pivot position inside the image (px)
18
19      def draw(self, target, pos, angle):
20          """Draw with the pivot at `pos`. angle (deg) turns the 'hanging down'
21          sprite counter-clockwise on screen, same as pygame.transform.rotate."""
22          # rotozoom rotates (smoothly) but makes a bigger image with the picture
23          # centred in it, so the pivot moves. Fix: take the vector from the pivot to
24          # the image centre, rotate that vector by the same angle, and put the
25          # rotated image's centre at pos + that vector -> the pivot ends at pos.
26          img = pygame.transform.rotozoom(self.image, angle, 1.0)
27          centre = pygame.Vector2(self.image.get_size()) / 2 - self.pivot
28          # Vector2.rotate turns the other way on screen (y points down), hence -angle
29          target.blit(img, img.get_rect(center=pos + centre.rotate(-angle)))
30
31      def flipped(self):
32          """Mirror image (left <-> right); the pivot is mirrored too."""
33          w = self.image.get_width()
34          return PivotSprite(pygame.transform.flip(self.image, True, False),
35                             (w - self.pivot.x, self.pivot.y))
36
37
38  def _crop_scale(surface, pivot, scale):
39      """Crop a canvas-sized png to its content and scale it; keep the pivot aligned."""
40      # bounding rect = smallest rectangle around the visible pixels (+2 px margin);
41      # cropping the empty canvas away makes rotating each frame much cheaper
42      rect = surface.get_bounding_rect(min_alpha=1).inflate(4, 4).clip(surface.get_rect())
43      part = surface.subsurface(rect).copy()
44      size = (max(1, round(rect.w * scale)), max(1, round(rect.h * scale)))
45      part = pygame.transform.smoothscale(part, size)
46      # the pivot was measured on the full canvas: shift it by the crop, then scale it
47      return PivotSprite(part, ((pivot[0] - rect.x) * scale, (pivot[1] - rect.y) * scale))
48
49
50  def stretch_leg(surface, k, split):
51      """Lengthen a leg png by k: rows between the pivot and `split` are stretched,
52      rows below `split` (the shoe) are only moved down. The pivot stays put."""
53      if k == 1:
54          return surface
55      w, h = surface.get_size()
56      top = int(config.LEG_PIVOT[1])
57      split = min(split, h)
58      mid_h = round((split - top) * k)  # new height of the stick part
59      out = pygame.Surface((w, top + mid_h + h - split), pygame.SRCALPHA)  # transparent canvas
60      out.blit(surface, (0, 0), (0, 0, w, top))  # 1) above the pivot: unchanged
61      mid = surface.subsurface((0, top, w, split - top))
62      out.blit(pygame.transform.smoothscale(mid, (w, mid_h)), (0, top))  # 2) the stick: stretched
63      out.blit(surface, (0, top + mid_h), (0, split, w, h - split))  # 3) the shoe: moved down
64      return out
65
66
67  def recolor_red(surface, weights):
68      """Filter that recolours the red shoe while keeping its shading.
69
70      On reddish pixels the 'redness' (R above the grey level) is redistributed

```

```

71     over the channels with `weights` (r, g, b), e.g. (0, 1, 0) = green.
72     """
73     out = surface.copy()
74     rgb = pygame.surfarray.pixels3d(out)      # numpy view of the pixels: writes change `out`
75     r, g, b = (rgb[..., i].astype(np.float32) for i in range(3))
76     mask = (r > g + 40) & (r > b + 40)        # "clearly red" pixels only (not white laces/black lines)
77     grey = (g + b) / 2                       # brightness without the red part
78     for i, w in enumerate(weights):
79         # new channel = grey + redness * weight -> bright red stays bright, dark stays dark
80         rgb[..., i][mask] = np.clip(grey + (r - grey) * w, 0, 255)[mask].astype(np.uint8)
81     del rgb                                  # release the pixel lock so the surface can be drawn
82     return out
83
84
85 class CharacterSprites:
86     """Head, thigh and shin(+shoe) sprites at game scale.
87
88     Leg sprites hang straight down from their pivot; shoes point right (+x),
89     so left legs use mirrored copies to keep the toes pointing outward.
90     """
91
92     def __init__(self, scale, leg_stretch=1.0):
93         def load(name):
94             return pygame.image.load(config.asset(name)).convert_alpha()
95
96         self.scale = scale                    # game px per png px
97         self.leg_stretch = leg_stretch        # 1.5 for the 1-player long legs
98         self.head = _crop_scale(load("head.png"), config.HEAD_CENTER, scale)
99         thigh = _crop_scale(stretch_leg(load("leg-part.png"), leg_stretch, config.SPRITE_CANVAS),
100                             config.LEG_PIVOT, scale)
101         shin = _crop_scale(stretch_leg(load("leg-part-with-shoe.png"), leg_stretch, config.SHOE_TOP),
102                             config.LEG_PIVOT, scale)
103         green = PivotSprite(recolor_red(shin.image, (0.0, 1.0, 0.0)), shin.pivot)
104         purple = PivotSprite(recolor_red(shin.image, (0.7, 0.0, 1.0)), shin.pivot)
105
106         # [side] -> sprite, side = -1 (left) or +1 (right)
107         self.thigh = {+1: thigh, -1: thigh.flipped()}
108         self.shin = {+1: shin, -1: shin.flipped()}          # normal: red shoe
109         self.shin_sticky = {+1: green, -1: green.flipped()} # sticky, in the air
110         self.shin_stuck = {+1: purple, -1: purple.flipped()} # sticky and glued to rock

```

inputs.py (67 lines)

The input interface and the keyboard input.

Lines	What happens there
1-7	Docstring and imports.
10-27	The interface: every input offers these methods.
30-45	Keyboard input: key layout and mouth keys.
47-49	Start every leg at its rest pose.
51-67	Turn held keys into angle changes, then hand each leg's pose to the player/limb that drives it.

Full source

```

1  """Input sources. The game only talks to InputSource, so camera and keyboard are swappable."""
2  import time
3
4  import pygame
5
6  import config
7  from player_state import LimbPose, PlayerState
8
9
10 class InputSource:
11     """The interface every input must offer. Subclasses override these methods;
12     the game calls them without knowing whether a camera or a keyboard is behind."""
13     num_players = 1
14
15     def set_num_players(self, n):
16         self.num_players = n
17
18     def get_players(self):
19         """List of PlayerState, one per player slot."""
20         raise NotImplementedError # a subclass must provide this
21
22     def get_preview(self):
23         """(RGB numpy frame or None, is_new) for the camera panel."""
24         return None, False
25
26     def close(self):
27         pass
28
29
30 class KeyboardInput(InputSource):
31     """Debug controls without a camera. Keys move a character leg directly; the
32     pose is handed to whichever player/limb drives that leg in the current scheme.
33
34     top-left    Q/A thigh, W/S knee    top-right    E/D thigh, R/F knee
35     bottom-left U/J thigh, I/K knee    bottom-right O/L thigh, P/; knee
36     hold 1-4 = that player's mouth open (sticky)
37     """
38     SPEED = 240.0 # deg/s
39     KEYS = { # leg -> (thigh +, thigh -, bend +, bend -)
40         "upper_left": (pygame.K_q, pygame.K_a, pygame.K_w, pygame.K_s),
41         "upper_right": (pygame.K_e, pygame.K_d, pygame.K_r, pygame.K_f),
42         "lower_left": (pygame.K_u, pygame.K_j, pygame.K_i, pygame.K_k),
43         "lower_right": (pygame.K_o, pygame.K_l, pygame.K_p, pygame.K_SEMICOLON),
44     }
45     MOUTH_KEYS = [pygame.K_1, pygame.K_2, pygame.K_3, pygame.K_4]
46
47     def __init__(self):
48         self.poses = {leg: LimbPose(*config.LEGS[leg][2:]) for leg in self.KEYS} # start at rest pose
49         self._t = time.time()
50
51     def get_players(self):
52         now = time.time()
53         step = self.SPEED * min(now - self._t, 0.1) # degrees to turn since last call
54         self._t = now
55         keys = pygame.key.get_pressed() # which keys are held right now
56         for leg, (tp, tm, bp, bm) in self.KEYS.items():
57             pose = self.poses[leg]
58             # True/False count as 1/0: (plus held) - (minus held) = +1, -1 or 0
59             pose.thigh += step * (keys[tp] - keys[tm])
60             pose.bend += step * (keys[bp] - keys[bm])
61
62         states = [PlayerState(visible=True, face_found=True, mouth_open=bool(keys[self.MOUTH_KEYS[i]]))
63                   for i in range(self.num_players)]
64         # hand each leg's pose to the player/limb that controls that leg
65         for player, limb, leg in config.CONTROL_SCHEMES[self.num_players]:
66             states[player].limbs[limb] = self.poses[leg]
67         return states

```

vision.py (381 lines)

Everything about the webcam: the background thread, both AI models and the per-player trackers (see chapter 9 and Part V).

Lines	What happens there
1-11	Docstring with the per-frame pipeline.
12-29	Imports.
30-39	One detected body: keypoints, confidences, box, face, player number.
40-44	Box size (closeness to camera).
45-50	Left/right position.
52-55	Average of visible head keypoints.
57-61	Body size for distance limits.
63-79	Square crop around the head for the face fallback.
82-98	Load YOLO on the fastest backend (TensorRT > GPU > CPU).
100-109	Run YOLO, convert results to Person objects.
112-132	Load MediaPipe (download the model if missing): VIDEO-mode for frames, IMAGE-mode for crops.
133-136	BGR numpy -> MediaPipe RGB image.
138-146	Faces on the whole frame, as pixel arrays.
148-162	Face in a head crop, mapped back to frame pixels.
164-167	Release the models.
170-189	Assign faces to bodies by pixel distance, closest first.
192-208	Number the biggest people left to right.
211-216	Lip gap / mouth width.
219-238	Smoothing, hysteresis and timeout for one player's mouth.
241-271	Ear-line roll angle -> sticky side, with a dead zone.
274-296	The camera input: models, camera, lock, thread start.
298-299	Ask the thread to switch player count.
301-311	Fresh trackers for a new player count.
312-330	The background loop: read, mirror, process, FPS, preview, store under the lock.
332-365	One frame -> PlayerStates.
366-368	Newest states (game thread).
370-375	Newest preview picture (game thread).
377-381	Stop the thread, release camera and models.

Full source

```

1  """Webcam -> players. YOLO pose for bodies, MediaPipe Face Landmarker for mouths.
2
3  Faces are found on the whole frame and given to the body whose head keypoints
4  are closest in pixels. Players are numbered left -> right on the mirrored image.
5  Everything runs in a background thread so the game keeps its frame rate.
6
7  Per camera frame (VisionInput._loop):
8      read frame -> mirror -> YOLO pose (bodies) -> pick/number players
9      -> MediaPipe faces -> match faces to bodies -> limb angles, mouth, head tilt
10     -> PlayerState per player + annotated preview picture for the game window
11 """
12 import os
13 import threading
14 import time
15 import urllib.request
16 from dataclasses import dataclass
17
18 import cv2
19 import mediapipe as mp
20 import numpy as np
21 from mediapipe.tasks.python import BaseOptions, vision as mp_vision
22
23 import config
24 from inputs import InputSource
25 from player_state import L_SH, R_SH, LIMBS, AngleSmoother, PlayerState, limb_poses
26 from preview import PreviewRenderer
27
28
29 @dataclass
30 class Person:
31     """One body found by the pose model in the current frame."""
32     xy: np.ndarray      # (17, 2) keypoints, pixels
33     conf: np.ndarray    # (17,)
34     box: np.ndarray     # x1, y1, x2, y2
35     face: np.ndarray | None = None # (478, 2) face landmarks, pixels
36     player: int | None = None
37
38     # @property: used like a value (p.area), computed when read
39     @property
40     def area(self):
41         """Size of the body's box; bigger = closer to the camera."""
42         return float((self.box[2] - self.box[0]) * (self.box[3] - self.box[1]))
43
44     @property
45     def center_x(self):
46         """Where the person stands left/right: middle of the shoulders (or of the box)."""
47         ok = self.conf[[L_SH, R_SH]] > config.KEYPOINT_THR
48         if ok.all():
49             return float(self.xy[[L_SH, R_SH], 0].mean())
50         return float((self.box[0] + self.box[2]) / 2)
51
52     def head_center(self):
53         """Average of the visible nose/eye/ear keypoints (0-4), or None."""
54         ok = self.conf[:5] > config.KEYPOINT_THR
55         return self.xy[:5][ok].mean(axis=0) if ok.any() else None
56
57     def shoulder_width(self):
58         """Body size in pixels, used to scale distance limits (far people are small)."""
59         if (self.conf[[L_SH, R_SH]] > config.KEYPOINT_THR).all():
60             return float(np.linalg.norm(self.xy[L_SH] - self.xy[R_SH]))
61         return float((self.box[2] - self.box[0]) / 2)
62
63     def head_box(self, shape):
64         """Square crop around nose/eyes/ears (fallback face search)."""
65         pts = self.xy[:5][self.conf[:5] > config.KEYPOINT_THR]
66         if len(pts) < 2:
67             return None
68         cx, cy = pts.mean(axis=0)
69         # span = how spread out the head keypoints are (np.ptp = max - min)
70         span = max(np.ptp(pts[:, 0]), np.ptp(pts[:, 1]), config.MIN_CROP / config.CROP_SCALE)

```

```

71     cy += config.CROP_DOWN * span                # eyes/ears sit above the mouth: shift down
72     half = span * config.CROP_SCALE / 2
73     h, w = shape[:2]
74     # clip the square to the picture borders
75     x1, y1 = int(max(cx - half, 0)), int(max(cy - half, 0))
76     x2, y2 = int(min(cx + half, w)), int(min(cy + half, h))
77     if x2 - x1 < config.MIN_CROP or y2 - y1 < config.MIN_CROP:
78         return None
79     return x1, y1, x2, y2
80
81
82 class PoseDetector:
83     """YOLO pose on the fastest backend available:
84     TensorRT engine (GPU) > PyTorch on GPU > PyTorch on CPU."""
85
86     def __init__(self, model_path=config.POSE_MODEL):
87         import torch                # slow imports, only needed in camera mode
88         from ultralytics import YOLO
89
90         gpu = torch.cuda.is_available()        # NVIDIA GPU usable?
91         engine = os.path.splitext(model_path)[0] + ".engine"    # yolo26n-pose.engine
92         if gpu and config.USE_TENSORRT and model_path.endswith(".pt") and os.path.exists(engine):
93             model_path, self.backend = engine, "TensorRT"
94         else:
95             self.backend = "GPU" if gpu else "CPU"
96             self.device = 0 if gpu else "cpu"
97             self.model = YOLO(model_path, task="pose")
98             print(f"Pose model: {model_path} on {self.backend}")
99
100    def detect(self, frame):
101        """All people in the frame as Person objects (keypoints + box)."""
102        r = self.model.predict(frame, verbose=False, device=self.device)[0]    # [0] = first (only) image
103        if r.keypoints is None or len(r.keypoints) == 0 or r.keypoints.conf is None:
104            return []
105        # results live on the GPU as torch tensors: .cpu().numpy() makes plain arrays
106        xy = r.keypoints.xy.cpu().numpy()
107        cf = r.keypoints.conf.cpu().numpy()
108        boxes = r.bboxes.xyxy.cpu().numpy()
109        return [Person(p, c, b) for p, c, b in zip(xy, cf, boxes)]
110
111
112 class FaceDetector:
113     """MediaPipe Face Landmarker: 478 points per face (eyes, lips, outline...)."""
114
115     def __init__(self, max_faces):
116         if not os.path.exists(config.FACE_MODEL):
117             print("Downloading face_landmarker.task ...")
118             urllib.request.urlretrieve(config.FACE_URL, config.FACE_MODEL)
119
120     def make(mode, n):
121         return mp_vision.FaceLandmarker.create_from_options(mp_vision.FaceLandmarkerOptions(
122             base_options=BaseOptions(model_asset_path=config.FACE_MODEL),
123             running_mode=mode, num_faces=n))
124
125     # VIDEO mode: for one continuous stream (it reuses the last frame's result
126     # to track faces faster). IMAGE mode: every picture is independent, needed
127     # for the head crops, which come from different people each time.
128     self.full = make(mp_vision.RunningMode.VIDEO, max_faces)
129     self.crop = make(mp_vision.RunningMode.IMAGE, 1) if config.FACE_CROP_FALLBACK else None
130     self._last_ts = -1
131
132     @staticmethod
133     def _mp_image(bgr):
134         # OpenCV pictures are BGR, MediaPipe wants RGB in one continuous memory block
135         rgb = np.ascontiguousarray(cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB))
136         return mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
137
138     def detect_full(self, frame):
139         """Faces on the whole frame, as (478, 2) arrays of pixel positions."""
140         # VIDEO mode needs a timestamp (ms) that always increases

```

```

141     ts = max(int(time.monotonic() * 1000), self._last_ts + 1)
142     self._last_ts = ts
143     res = self.full.detect_for_video(self._mp_image(frame), ts)
144     h, w = frame.shape[:2]
145     # landmarks come as 0..1 fractions of the picture: multiply by its size
146     return [np.array([(p.x * w, p.y * h) for p in face]) for face in res.face_landmarks]
147
148 def detect_crop(self, frame, box):
149     """Face inside one head crop (fallback for faces too small on the full frame)."""
150     if self.crop is None:
151         return None
152     x1, y1, x2, y2 = box
153     crop = frame[y1:y2, x1:x2]
154     ch, cw = crop.shape[:2]
155     scale = max(1.0, config.UPSCALE_TO / max(ch, cw))
156     if scale > 1.0: # enlarge tiny crops so the model sees detail
157         crop = cv2.resize(crop, None, fx=scale, fy=scale, interpolation=cv2.INTER_LINEAR)
158     res = self.crop.detect(self._mp_image(crop))
159     if not res.face_landmarks:
160         return None
161     # crop fractions -> crop pixels -> full-frame pixels (add the crop's corner)
162     return np.array([(p.x * cw + x1, p.y * ch + y1) for p in res.face_landmarks[0]])
163
164 def close(self):
165     self.full.close()
166     if self.crop:
167         self.crop.close()
168
169 def match_faces(persons, faces):
170     """Give each face to the person whose head keypoints are closest (greedy)."""
171     # 1) every (person, face) pair that is close enough, with its pixel distance
172     pairs = []
173     for i, p in enumerate(persons):
174         head = p.head_center()
175         if head is None:
176             continue
177         limit = max(config.FACE_MATCH_MIN_PX, config.FACE_MATCH_FACTOR * p.shoulder_width())
178         for j, f in enumerate(faces):
179             d = float(np.linalg.norm(f.mean(axis=0) - head)) # face centre <-> head centre
180             if d < limit:
181                 pairs.append((d, i, j))
182     # 2) closest pairs first; each person and each face is used at most once
183     used_p, used_f = set(), set()
184     for _, i, j in sorted(pairs):
185         if i not in used_p and j not in used_f:
186             persons[i].face = faces[j]
187             used_p.add(i)
188             used_f.add(j)
189
190
191 def assign_players(persons, frame_w, n):
192     """Pick the n biggest people and number them left -> right on screen."""
193     chosen = sorted(persons, key=lambda p: -p.area)[:n] # biggest = closest = the players
194     chosen.sort(key=lambda p: p.center_x) # then order them left to right
195     if len(chosen) == n:
196         slots = list(range(n))
197     else: # fewer people than players: use the screen region they stand in
198         slots, taken = [], set()
199         for p in chosen:
200             # split the picture into n equal strips; the strip you stand in = your number
201             s = min(n - 1, max(0, int(p.center_x / frame_w * n)))
202             while s in taken:
203                 s = (s + 1) % n
204             taken.add(s)
205             slots.append(s)
206         for p, s in zip(chosen, slots):
207             p.player = s
208
209
210

```

```

211 def mouth_ratio(pts):
212     """Lip gap / mouth width. Face-mesh points 13/14 = middle of the upper/lower
213     lip, 61/291 = the mouth corners. Dividing by the width makes the number the
214     same whether the face is near (big) or far (small) from the camera."""
215     width = np.linalg.norm(pts[61] - pts[291])
216     return float(np.linalg.norm(pts[13] - pts[14]) / width) if width else 0.0
217
218
219 class MouthTracker:
220     """Smoothed mouth opening with hysteresis, per player slot."""
221
222     def __init__(self):
223         self.score, self.open, self.last_seen = 0.0, False, 0.0
224
225     def update(self, pts, now):
226         if pts is None:
227             # face not found this frame
228             self.open, self.score = False, 0.0
229             return
230             # short drop-outs keep the last state
231         self.last_seen = now
232         # smoothed score (EMA): half new measurement, half previous value
233         self.score = config.MOUTH_EMA * mouth_ratio(pts) + (1 - config.MOUTH_EMA) * self.score
234         # Hysteresis: open above 0.35, closed below 0.25, in between keep the state.
235         # One single threshold would flicker open/closed when the score hovers near it.
236         if self.open and self.score < config.MOUTH_CLOSE_T:
237             self.open = False
238         elif not self.open and self.score > config.MOUTH_OPEN_T:
239             self.open = True
240
241 class HeadTilt:
242     """Which side sticks, from the sideways tilt (roll) of the head.
243
244     roll = angle of the line from the screen-left ear to the screen-right ear
245     (the eyes are used if an ear is not seen). On the mirrored image the
246     screen-right ear is the player's right ear, so roll > 0 means the right
247     ear is lower = head tilted to the right. Tilt right -> right side, tilt
248     left -> left side (HEAD_TILT_INVERT swaps this). Inside +-HEAD_TILT_ANGLE
249     the last side is kept.
250     """
251     PAIRS = ((3, 4), (1, 2)) # ears, then eyes
252
253     def __init__(self):
254         self.side = "right"
255         self.roll = 0.0
256
257     def update(self, person):
258         ok = person.conf > config.KEYPOINT_THR
259         for a, b in self.PAIRS:
260             if ok[a] and ok[b]:
261                 left, right = sorted((person.xy[a], person.xy[b]), key=lambda p: p[0])
262                 # slope angle of the line left ear -> right ear (0 = level head)
263                 self.roll = float(np.degrees(np.arctan2(right[1] - left[1], right[0] - left[0])))
264                 break
265             else:
266                 # for-else: runs when the loop found no usable pair
267                 return self.side
268         tilt_right = self.roll > config.HEAD_TILT_ANGLE
269         tilt_left = self.roll < -config.HEAD_TILT_ANGLE
270         if tilt_right or tilt_left:
271             self.side = "right" if tilt_right != config.HEAD_TILT_INVERT else "left"
272         return self.side
273
274 class VisionInput(InputSource):
275     """Camera input. A background thread does all detection; the game thread only
276     picks up the newest results. A lock makes sure the game never reads a result
277     while the thread is half-way through replacing it."""
278
279     def __init__(self, source=config.CAMERA_SOURCE, pose_model=config.POSE_MODEL):
280         self.pose = PoseDetector(pose_model)

```

```

281     self.faces = FaceDetector(config.MAX_PLAYERS + 2)
282     self.renderer = PreviewRenderer()
283     PreviewRenderer.backend = self.pose.backend
284     self.cap = cv2.VideoCapture(int(source) if str(source).isdigit() else source)
285     if not self.cap.isOpened():
286         raise SystemExit(f"Could not open camera/video source {source!r}")
287
288     self._lock = threading.Lock()
289     self._preview = None
290     self._new_preview = False
291     self._reset_players(self.num_players)
292     self._wanted_n = self.num_players
293     self._running = True
294     # daemon thread: does not keep Python alive when the game window closes
295     self._thread = threading.Thread(target=self._loop, daemon=True)
296     self._thread.start()
297
298     def set_num_players(self, n):
299         self._wanted_n = n          # picked up by the worker thread
300
301     def _reset_players(self, n):
302         """Fresh per-player trackers for a new player count (and its control scheme)."""
303         self.n = n
304         self.scheme = config.CONTROL_SCHEMES[n]
305         self.smoothers = [{limb: AngleSmoother() for limb in LIMBS} for _ in range(n)]
306         self.mouths = [MouthTracker() for _ in range(n)]
307         self.tilts = [HeadTilt() for _ in range(n)] if n in config.HEAD_TILT_PLAYERS else None
308         with self._lock:
309             self._states = [PlayerState() for _ in range(n)]
310
311     # ----- worker thread -----
312     def _loop(self):
313         fps, t_prev = 0.0, time.time()
314         while self._running:
315             if self._wanted_n != self.n:
316                 self._reset_players(self._wanted_n)
317             ok, frame = self.cap.read()
318             if not ok:
319                 time.sleep(0.01)
320                 continue
321             if config.MIRROR_CAMERA:
322                 frame = cv2.flip(frame, 1)          # 1 = flip left/right, like a mirror
323             states, persons = self._process(frame)
324             now = time.time()
325             fps = 0.9 * fps + 0.1 / max(now - t_prev, 1e-6)  # smoothed frames per second
326             t_prev = now
327             preview = cv2.cvtColor(self.renderer.draw(frame, persons, states, self.scheme, fps),
328                                   cv2.COLOR_BGR2RGB)
329             with self._lock:
330                 self._states, self._preview, self._new_preview = states, preview, True
331
332     def _process(self, frame):
333         """One camera frame -> one PlayerState per player slot."""
334         now = time.time()
335         persons = self.pose.detect(frame)           # bodies
336         assign_players(persons, frame.shape[1], self.n)  # who is P1, P2, ...
337         match_faces(persons, self.faces.detect_full(frame))  # faces -> bodies
338
339         states = [PlayerState() for _ in range(self.n)]
340         by_slot = {p.player: p for p in persons if p.player is not None}
341         for slot in range(self.n):
342             p = by_slot.get(slot)
343             st = states[slot]
344             if p is not None:
345                 if p.face is None:                  # retry on a head crop
346                     box = p.head_box(frame.shape)
347                     if box is not None:
348                         p.face = self.faces.detect_crop(frame, box)
349                 poses = limb_poses(p.xy, p.conf)
350                 st.visible = True

```

```

351         for limb, smoother in self.smoothers[slot].items():
352             value = smoother.update(poses.get(limb))
353             if value is not None:
354                 st.limbs[limb] = value
355             st.face_found = p.face is not None
356         if self.tilts is not None:
357             tilt = self.tilts[slot]
358             st.sticky_side = tilt.update(p) if p is not None else tilt.side
359             st.head_tilt = tilt.roll
360         mouth = self.mouths[slot]
361         mouth.update(p.face if p is not None else None, now)
362         st.mouth_open, st.mouth_score = mouth.open, mouth.score
363     return states, persons
364
365     # ----- main thread -----
366     def get_players(self):
367         with self._lock: # wait if the thread is writing right now
368             return list(self._states)
369
370     def get_preview(self):
371         """Newest annotated camera frame (RGB numpy array) and whether it is new."""
372         with self._lock:
373             frame, fresh = self._preview, self._new_preview
374             self._new_preview = False
375         return frame, fresh
376
377     def close(self):
378         self._running = False
379         self._thread.join(timeout=2)
380         self.cap.release()
381         self.faces.close()

```

player_state.py (108 lines)

Data passed from input to game, and the arm-angle maths (see chapter 9).

Lines	What happens there
1-9	Docstring with the 17 COCO keypoints.
10-25	Imports and keypoint index constants.
26-32	Two angles from one limb.
33-41	Everything known about one player this frame.
44-53	Angle helpers.
56-62	Which keypoints form which limb, by screen side.
65-90	Torso direction and the two angles for every visible limb.
91-108	Wrap-safe exponential moving average.

Full source

```

1  """What the game needs to know about each human player, plus the limb-angle math.
2
3  The pose model gives 17 "keypoints" per person in COCO order:
4      0 nose, 1 left eye, 2 right eye, 3 left ear, 4 right ear,
5      5 left shoulder, 6 right shoulder, 7 left elbow, 8 right elbow,
6      9 left wrist, 10 right wrist, 11 left hip, 12 right hip,
7      13 left knee, 14 right knee, 15 left ankle, 16 right ankle
8  Here they are turned into two angles per limb, which drive one character leg.
9  """
10 import math
11 from dataclasses import dataclass, field
12
13 import numpy as np
14
15 import config
16
17 # COCO keypoints used here
18 L_SH, R_SH, L_HIP, R_HIP = 5, 6, 11, 12
19 ARM_CHAINS = [(5, 7, 9), (6, 8, 10)] # shoulder, elbow, wrist
20 LEG_CHAINS = [(11, 13, 15), (12, 14, 16)] # hip, knee, ankle
21 LIMBS = ("left_arm", "right_arm", "left_leg", "right_leg")
22
23
24 # @dataclass writes __init__ and printing for us: LimbPose(30.0, -10.0)
25 @dataclass
26 class LimbPose:
27     """Leg command from one body limb. Degrees, 'outward' convention (see config.LEGS)."""
28     thigh: float # shoulder/hip angle vs torso: 0 = hanging down, + = away from body
29     bend: float # elbow/knee angle: lower segment direction - upper segment direction
30
31
32 @dataclass
33 class PlayerState:
34     """Everything the game reads about one player, refreshed every camera frame."""
35     visible: bool = False
36     limbs: dict = field(default_factory=dict) # limb name -> LimbPose
37     mouth_open: bool = False
38     mouth_score: float = 0.0
39     face_found: bool = False
40     sticky_side: str | None = None # "left"/"right": only that side's limb sticks, None = all
41     head_tilt: float = 0.0 # deg, sideways head roll; > 0 = tilted to the right (1 player)
42
43
44 def _heading(v):
45     """Angle of a screen vector measured from 'down', positive towards +x."""
46     # atan2(x, y) instead of the usual atan2(y, x): measures from straight down
47     # (screen y grows downward), so an arm hanging down gives 0 degrees.
48     return math.degrees(math.atan2(v[0], v[1]))
49
50
51 def _wrap(a):
52     """Bring an angle into -180..180."""
53     return (a + 180.0) % 360.0 - 180.0
54
55
56 def limb_chains(xy):
57     """limb name -> keypoint chain, by screen side (left = smaller x)."""
58     # Sides come from where the joints are on screen, not from the model's
59     # "left/right" labels: on a mirrored picture those labels can be swapped.
60     arms = sorted(ARM_CHAINS, key=lambda c: xy[c[0]][0])
61     legs = sorted(LEG_CHAINS, key=lambda c: xy[c[0]][0])
62     return dict(zip(LIMBS, arms + legs))
63
64
65 def limb_poses(xy, conf, thr=config.KEYPOINT_THR):
66     """limb name -> LimbPose for every limb whose three keypoints are visible."""
67     ok = conf > thr # True for every keypoint the model is sure enough about
68     if not (ok[L_SH] and ok[R_SH]):
69         return {}
70     # torso axis: shoulders -> hips, or straight down if hips are out of frame

```

```

71     if ok[L_HIP] and ok[R_HIP]:
72         down = (xy[L_HIP] + xy[R_HIP]) / 2 - (xy[L_SH] + xy[R_SH]) / 2
73     else:
74         down = np.array([0.0, 1.0])
75     torso = _heading(down) # leaning the whole body should not swing the legs
76
77     out = {}
78     for name, (a, b, c) in limb_chains(xy).items(): # a = shoulder, b = elbow, c = wrist
79         if not (ok[a] and ok[b] and ok[c]):
80             continue
81         # side = -1 for left limbs: mirror the angles so "away from the body" is
82         # positive on both sides (the same convention as the character's legs)
83         side = -1 if name.startswith("left") else 1
84         upper = side * _wrap(_heading(xy[b] - xy[a]) - torso) # upper arm direction vs torso
85         lower = side * _wrap(_heading(xy[c] - xy[b]) - torso) # forearm direction vs torso
86         out[name] = LimbPose(upper, _wrap(lower - upper)) # bend = forearm relative to upper arm
87     return out
88
89
90 @dataclass
91 class AngleSmoother:
92     """EMA on angles (wrap-safe) so jittery keypoints don't shake the legs."""
93     alpha: float = config.ANGLE_EMA
94     value: LimbPose | None = None
95
96     def update(self, new):
97         """Exponential moving average: move a fraction `alpha` of the way to the new
98         measurement. Wrapping the difference keeps 179 -> -179 a 2 degree step,
99         not a 358 degree swing. A missing measurement keeps the last value."""
100         if new is None:
101             return self.value
102         if self.value is None:
103             self.value = LimbPose(new.thigh, new.bend)
104         else:
105             self.value = LimbPose(
106                 _wrap(self.value.thigh + self.alpha * _wrap(new.thigh - self.value.thigh)),
107                 _wrap(self.value.bend + self.alpha * _wrap(new.bend - self.value.bend))
108             )
109         return self.value

```

preview.py (85 lines)

Draws the annotated camera picture.

Lines	What happens there
1-7	Docstring and imports.
9-15	Which points to connect: body skeleton, lips, leg labels.
18-60	Grey skeleton for everybody, controlling limbs thick and coloured, player label, lips.
61-68	Lip outline, yellow and thicker when open.
69-85	FPS line and one status line per player.

Full source

```

1  """Draws the camera preview: grey bodies, controlling limbs in the player's colour, mouths only."""
2  import cv2
3  import numpy as np
4  from mediapipe.tasks.python.vision import FaceLandmarksConnections
5
6  import config
7  from player_state import limb_chains
8
9  # pairs of COCO keypoints joined by a line (the stick figure)
10 BODY_EDGES = [(0, 1), (0, 2), (1, 3), (2, 4), (5, 6), (5, 11), (6, 12), (11, 12),
11               (5, 7), (7, 9), (6, 8), (8, 10), (11, 13), (13, 15), (12, 14), (14, 16)]
12 # pairs of face-mesh points that outline the lips (MediaPipe provides this list)
13 LIPS = [(c.start, c.end) for c in FaceLandmarksConnections.FACE_LANDMARKS_LIPS]
14 LEG_LABELS = {"upper_left": "top-left", "upper_right": "top-right",
15              "lower_left": "bottom-left", "lower_right": "bottom-right"}
16
17
18 class PreviewRenderer:
19     backend = "" # shown next to the FPS, e.g. "TensorRT"
20
21     def draw(self, frame, persons, states, scheme, fps):
22         img = frame.copy() # never draw on the frame the detectors use
23         ok_thr = config.KEYPOINT_THR
24         for p in persons:
25             ok = p.conf > ok_thr
26             pts = p.xy.astype(int) # OpenCV draws at whole pixels
27             grey = config.BODY_COLOR_BGR
28             # 1) the whole body in grey
29             for i, j in BODY_EDGES:
30                 if ok[i] and ok[j]:
31                     cv2.line(img, tuple(pts[i]), tuple(pts[j]), grey, 2, cv2.LINE_AA)
32             for k in np.nonzero(ok)[0]:
33                 cv2.circle(img, tuple(pts[k]), 3, grey, -1)
34
35             # 2) on top: only the limbs this player controls, thick and coloured
36             if p.player is not None:
37                 col = config.PLAYER_COLORS_BGR[p.player]
38                 chains = limb_chains(p.xy)
39                 for player, limb, _ in scheme:
40                     if player != p.player:
41                         continue
42                     a, b, c = chains[limb]
43                     for i, j in ((a, b), (b, c)):
44                         if ok[i] and ok[j]:
45                             cv2.line(img, tuple(pts[i]), tuple(pts[j]), col, 5, cv2.LINE_AA)
46                     for k in (a, b, c):
47                         if ok[k]:
48                             cv2.circle(img, tuple(pts[k]), 5, col, -1)
49                 head = p.head_center()
50                 if head is not None:
51                     cv2.putText(img, f"P{p.player + 1}", (int(head[0]) - 15, int(head[1]) - 50),
52                                cv2.FONT_HERSHEY_SIMPLEX, 0.8, col, 2)
53             # 3) the mouth outline (the only part of the face mesh that is shown)
54             if p.face is not None:
55                 self._draw_mouth(img, p, states)
56
57         self._draw_hud(img, states, scheme, fps)
58         return img
59
60     @staticmethod
61     def _draw_mouth(img, p, states):
62         face = p.face.astype(int)
63         is_open = p.player is not None and states[p.player].mouth_open
64         col = (0, 255, 255) if is_open else (255, 255, 255) # yellow when open (BGR)
65         for i, j in LIPS:
66             cv2.line(img, tuple(face[i]), tuple(face[j]), col, 2 if is_open else 1, cv2.LINE_AA)
67
68     @staticmethod
69     def _draw_hud(img, states, scheme, fps):
70         """Text in the top-left corner: FPS and one status line per player."""

```

```

71     y = 25
72     cv2.putText(img, f"{fps:.0f} FPS {PreviewRenderer.backend}", (10, y),
73                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
74     for i, st in enumerate(states):
75         y += 25
76         legs = ", ".join(LEG_LABELS[leg] for pl, _, leg in scheme if pl == i)
77         if not st.visible:
78             status = "not found"
79         else:
80             mouth = "OPEN (sticky)" if st.mouth_open else "closed"
81             status = f"mouth {mouth}" if st.face_found else "no face"
82             if st.sticky_side is not None:
83                 status += f" | sticks {st.sticky_side.upper()} (roll {st.head_tilt:+.0f} deg)"
84             cv2.putText(img, f"P{i + 1} [{legs}]: {status}", (10, y),
85                         cv2.FONT_HERSHEY_SIMPLEX, 0.55, config.PLAYER_COLORS_BGR[i], 2)

```

export_engine.py (17 lines)

Builds the TensorRT engine for the pose model (see Part V).

Lines	What happens there
1-8	Docstring: when and why to run it.
9-11	Imports.
13-17	Export with half precision at 640 px on GPU 0.

Full source

```

1  """Build the TensorRT engine for the pose model (takes a few minutes, needs an NVIDIA GPU).
2
3      python export_engine.py
4
5  The .engine only works on the GPU / driver / TensorRT version it was built with,
6  so run this again on another computer or after updating them.
7  Needs: pip install tensorrt-cu13 (and the CUDA build of torch)
8  """
9  from ultralytics import YOLO
10
11  import config
12
13  if __name__ == "__main__":
14      # format="engine": TensorRT. quantize=16: half-precision numbers (faster, tiny
15      # accuracy loss). imgsz=640: the picture size the network works on. device=0: first GPU.
16      path = YOLO(config.POSE_MODEL).export(format="engine", quantize=16, imgsz=640, device=0)
17      print("saved", path)

```

Part IV - Reference

12. Every setting in config.py

Read straight from the file when this guide was built. Grey rows are the section headers of the file.

Setting	Value	Meaning
ROOT	<code>os.path.dirname(os.path.abspath(__file__))</code>	
ASSET_DIR	<code>os.path.join(ROOT, "assets")</code>	
[maps]		
MAP_HEIGHT	2164	
MAPS	(several lines - see the listing)	
SPAWN_LIFT	1 / 8	start this far (MAP_HEIGHT fraction) above the floor, or mid-gap if lower
[map / view]		
VIEW_FRACTION	1 / 3	game view height = 1/3 of the map height
VIEW_ASPECT	4096 / 2164	view width / height (the mountain map's shape)
WINDOW_SCALE	1.0	max scale of the window (it also shrinks to fit the screen)
LAYOUT_MARGIN	0.025	gap between window edge / game / camera
LAYOUT_CAMERA_WIDTH	0.40	camera panel width
LAYOUT_SCREEN_FILL	0.92	window may use at most this fraction of the desktop
LAYOUT_BG	(40, 42, 48)	dark grey around the two panels
START_FULLSCREEN	False	F11 toggles fullscreen at any time
FPS	60	frames drawn per second (the loop waits to keep this rate)
PHYSICS_SUBSTEPS	3	physics steps per rendered frame (stability)
CAMERA_LERP	6.0	how fast the game view follows the character (1/s)
SKY_COLOR	(135, 206, 235)	fills transparent gaps behind the map images
[character size]		
CHARACTER_FRACTION	1 / 8	
SPRITE_CANVAS	500	size of the character png canvases
HEAD_CENTER	(250, 250)	
HEAD_RADIUS	95	collision circle of the head
LEG_PIVOT	(249.5, 190.5)	joint at the top of leg-part.png / leg-part-with-shoe.png
SEGMENT_LENGTH	117	pivot -> pivot of leg-part.png
FOOT_OFFSET	(20, 160)	shoe centre from LEG_PIVOT while the sprite hangs down (toe = +x)
FOOT_RADIUS	30	collision circle of the shoe
LEGS	(several lines - see the listing)	
CONTROL_SCHEMES	(several lines - see the listing)	

Setting	Value	Meaning
MAX_PLAYERS	4	highest player count offered in the menu
LEG_STRETCH	{1: 1.5}	
SHOE_TOP	300	canvas y where the shoe starts on leg-part-with-shoe.png
[flag]		
FLAG_SEARCH	0.1	fraction of the map width searched for the peak
FLAG_FRACTION	1 / 9	flag.png canvas height relative to MAP_HEIGHT
FLAG_BASE	(160, 492)	canvas px of flag.png that sits on the ground (pole foot)
[physics]		
GRAVITY	2000.0	px/s ² (map pixels)
MAX_SPEED	1500.0	px/s, clamp on body velocity
AIR_DRAG	0.3	1/s
LEG_MAX_SPEED	420.0	deg/s, how fast a leg joint can follow the arm
SOLVER_ITERATIONS	4	passes over all contacts per physics step (more = stiffer, slower)
WALKABLE_SLOPE	55.0	deg; feet only get friction on surfaces flatter than this
CONTACT_EPS	2.0	px; a foot this close to the ground still counts as touching
STICKY_GRAB_DIST	8.0	px; a sticky foot grabs ground within this distance
STICKY_BREAK_DIST	40.0	px; a sticky foot tears off if pulled this far from its anchor
STICKY_STIFFNESS	0.6	share of a glued foot's error fixed per solver pass (1 = all)
LEG_BLOCK_TOLERANCE	3.0	px a free foot may sink into rock while another foot is glued
STICKY_SINK	6.0	px a sticky/glued foot may sink into rock while another foot is glued
HEAD_FRICTION	6.0	1/s, damping while the head scrapes the ground
[vision]		
CAMERA_SOURCE	"0"	webcam index or video path
POSE_MODEL	"yolo26n-pose.pt"	YOLO "nano" pose model, downloaded automatically
USE_TENSORRT	True	
FACE_MODEL	os.path.join(ROOT, "face_landmarker.task")	MediaPipe face model file
FACE_URL	("https://storage.googleapis.com/mediapipe-mod...	
MIRROR_CAMERA	True	flip the webcam so it behaves like a mirror
KEYPOINT_THR	0.5	keypoints with lower confidence (0..1) count as "not seen"
FACE_MATCH_FACTOR	1.0	max face-to-head distance = factor * shoulder width
FACE_MATCH_MIN_PX	40	
FACE_CROP_FALLBACK	True	if the full-frame face search misses a player, retry on a head crop

Setting	Value	Meaning
CROP_SCALE	2.4	head crop size = spread of nose/eyes/ears * this
CROP_DOWN	0.25	move the crop down (in spreads) so chin and mouth are inside
MIN_CROP	24	px; smaller heads are skipped (too few pixels for a face mesh)
UPSCALE_TO	256	small crops are enlarged to this before the face model
MOUTH_OPEN_T, MOUTH_CLOSE_T	0.35, 0.25	hysteresis on lip gap / lip width
MOUTH_EMA	0.5	smoothing of the mouth score (1 = no smoothing)
MOUTH_TIMEOUT	0.5	s; forget the mouth state if the face is lost for this long
HEAD_TILT_PLAYERS	{1}	player counts that use this
HEAD_TILT_ANGLE	10.0	deg the ear line must tilt to switch side (in between: keep last)
HEAD_TILT_INVERT	False	True: swap (tilt right -> left-arm leg)
ANGLE_EMA	0.6	smoothing of arm angles (1 = no smoothing)
PLAYER_COLORS_BGR	[(0, 0, 255), (0, 220, 0), (255, 120, 0), (0, ...	
PLAYER_COLORS_RGB	[tuple(reversed(c)) for c in PLAYER_COLORS_BGR...	
BODY_COLOR_BGR	(150, 150, 150)	grey for non-controlling body parts in the preview
[menu]		
GAME_TITLE	"All For One"	
GAME_CREDIT	"From The Group MeEeEe"	
GAME_CREDIT_EMOJI	"[sheep emoji]"	sheep

13. How do I...?

I want to...	Change this
add a map	Put background-x.png and foreground-x.png (transparent = air) in assets/ and add an entry to MAPS in config.py with a spawn point (fractions of the map) and optionally a scale . The flag and the menu picture are automatic.
make jumps stronger / weaker	LEG_MAX_SPEED (how fast legs straighten) or GRAVITY .
change who controls which leg	CONTROL_SCHEMES - each row is (player, limb, leg).
make the mouth easier to trigger	Lower MOUTH_OPEN_T (and keep MOUTH_CLOSE_T about 0.1 lower).
make legs smoother / more responsive	ANGLE_EMA (lower = smoother, slower).
change the character size	CHARACTER_FRACTION (relative to the map height).
make a map bigger relative to the character	That map's "scale" in MAPS .
change the player colours	PLAYER_COLORS_BGR (blue, green, red order!).
switch head-tilt direction	HEAD_TILT_INVERT .
start in fullscreen	START_FULLSCREEN = True .
use the camera without mirroring	MIRROR_CAMERA = False .

14. Glossary

Word	Meaning here
anchor	The point where a sticky foot is glued to the rock.
bilinear interpolation	Blending the 4 nearest table values by distance, for smooth results between pixels.
blit	Copy (paste) one picture onto another (pygame).
canvas	The fixed-size picture every scene draws on.
delta time (dt)	Seconds since the last frame/step; movement = speed x dt.
EMA	Exponential moving average: smooth = smooth + a x (new - smooth).
Gauss-Seidel	Fixing constraints one after another, repeated a few times.
hysteresis	Two thresholds (on/off) with a gap so a value near the edge does not flicker.
inference	Running a trained AI model on new input.
keypoint / landmark	A named point found by a model (body joint / face point).
kinematics	Computing joint positions from angles and lengths.
lock	Makes one thread wait while another changes shared data.
normal	Unit vector pointing straight out of a surface.
PBD	Position-based dynamics: move, correct positions, derive velocity.
SDF	Signed distance field: distance to the nearest surface, negative inside.
substep	One of several smaller physics steps inside one frame.
tangent	Direction along a surface (normal turned 90 degrees).
thread	A part of the program that runs at the same time as others.
TensorRT / FP16	NVIDIA's optimiser for running networks on its GPUs / 16-bit numbers.

Part V - How the AI models work

This part explains the two models at the level needed to use them well, without the mathematics of training.

15. Pose detection: YOLO

What it does. Given a picture, `yolo26n-pose` returns for every person a box, a person-confidence and 17 keypoints (x, y and a confidence 0..1 for each). It is a *neural network*: millions of numbers (weights) learned by training on the COCO dataset, where people were labelled by hand with these 17 points.

How it works, roughly. YOLO stands for *You Only Look Once*: the whole picture goes through the network a single time and all people come out together, which is why it is fast enough for live video.

1. The frame is resized to fit 640 x 640 (padding the rest) and turned into numbers.
2. The *backbone* (stacked convolution layers) turns the picture into feature maps: coarse grids where each cell describes what is in its region - edges first, then shapes, then body parts.
3. The *neck* combines feature maps of different sizes, so both near (big) and far (small) people are found.
4. The *head* predicts, for each grid cell, whether a person is centred there, the box, and the 17 keypoint positions with their confidences.
5. Overlapping duplicates are removed, leaving one result per person. (Ultralytics designs the YOLO26 family to do this inside the network, without a separate clean-up step.)

The **n** means *nano*, the smallest and fastest size. Bigger sizes (s, m, l, x) are more accurate on small or far people but slower. **Keypoint confidence** drops for hidden or blurry joints; the game ignores keypoints below 0.5, and a limb with a missing joint simply keeps its last angle.

Speed: CPU, GPU and TensorRT

A network is mostly multiplication of large tables of numbers, which a graphics card does in parallel. With the CUDA build of PyTorch the model runs on the RTX GPU. **TensorRT** goes further: it compiles the network for *this specific* GPU - merging layers, choosing the fastest routines and using 16-bit numbers (FP16) instead of 32-bit. That is why the `.engine` file only works on the computer it was built on.

Backend	Pose model per frame	Whole vision step per frame
CPU	about 51 ms	about 110 ms (9 FPS)
GPU (CUDA PyTorch)	about 12 ms	about 33 ms
TensorRT FP16	about 5 ms	about 23 ms (40+ FPS)

Measured on the development laptop (RTX 4070 Laptop) on a sample photo; the rest of the vision step is mostly the face model, which runs on the CPU.

16. Face landmarks: MediaPipe Face Landmarker

What it does. For every face it returns **478 landmarks**: 468 points of a face mesh (outline, eyebrows, eyes, nose, lips) plus 10 iris points, each as a position relative to the picture. It can also return 52 *blendshape* scores such as `jawOpen` (your original test script printed this); the game uses the simpler lip-gap ratio instead, because it also works on the head-crop fallback.

How it works. It is two networks in a row:

1. A small, very fast **face detector** (BlazeFace) finds where faces are and how they are rotated.
2. Each face is cut out, straightened and resized, and a **landmark network** predicts the positions of all 478 points at once (it *regresses* the coordinates: the output layer simply is the list of x, y, z values).

In **VIDEO mode** (used on the full frame) it *tracks*: the landmarks of the previous frame tell it where the face is now, so the detector can often be skipped - faster and steadier. That is why it needs increasing timestamps. **IMAGE mode** (used for head crops) treats every picture independently, which is right there because consecutive crops can come from different people.

17. Limits to keep in mind

- **Distance**: the face model is meant for faces within about two metres; far away, faces get too small. The head-crop fallback enlarges small faces, which helps.
- **Light and blur**: dark rooms and fast arm movement lower keypoint confidence; limbs then hold their last angle.
- **Overlap**: players standing behind each other can swap numbers or hide limbs; standing side by side works best.
- **Side view**: a head turned far sideways hides one ear and much of the mouth; head tilt falls back to the eyes, the mouth may be lost briefly (kept for 0.5 s).
- **Mirroring**: the model's own left/right labels can be wrong on a mirrored picture - the code uses screen positions instead (chapter 9).

Everything in this guide refers to the code as it is in the repository at the time of writing. The tables in Part III find their line numbers automatically from the real files, so they match the listings.